

A survey of large-page support

Ian Wienand

Document Number: ERTOS 10100:2006

Copyright (C) 2006 The University of New South Wales

This work is sponsored by the University of New South Wales, National ICT Australia, Gelato.Org, HP, Australian Research Council.

THIS DOCUMENT IS PROVIDED “AS IS” WITHOUT ANY WARRANTIES, INCLUDING ANY WARRANTY OF MERCHANTABILITY, NON-INFIRNGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, OR ANY WARRANTY OTHERWISE ARISING OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

Permission to make digital or hard copies of this work for personal or commercial use, including redistribution, is granted without fee, provided that the copies are distributed in tact, without any deletions, alterations or additions. In particular, this copyright notice and the authorship must be preserved on all copies. To copy otherwise, or to modify, requires prior specific permission.

Contact Details:

Gelato@UNSW
Attention: Ian Wienand

Embedded, Real-Time and Operating Systems
Locked Bag 6016
The University of New South Wales
Sydney NSW 1466

email: **ianw@gelato.unsw.edu.au**
web: **http://www.gelato.unsw.edu.au/**

Contents

1 Motivation **5**

1.1 Virtual Memory and the TLB 5
1.2 Increasing TLB coverage 5
1.2.1 More TLB entries 5
1.2.2 Larger Pages 6
1.3 Multiple Page Sizes 7
1.3.1 Superpages defined 8
1.4 Overview 8

2 Hardware support for multiple page sizes **9**

2.1 Hardware constraints 9
2.1.1 Set-associativity 9
2.1.2 Discussion 10
2.2 Hardware-Based Approach 11
2.2.1 Sub-blocking 11
2.2.2 Skewed TLB 13
2.2.3 Zip Code TLB 13
2.3 Software Approaches 14
2.3.1 Software-managed address translation 14
2.4 Multiple Page-Size Support in current processors 15
2.4.1 Alpha Processor 16
2.4.2 MIPS R10000 16
2.4.3 SPARC Processor 16
2.4.4 ARM Processor 18
2.4.5 Itanium 19
2.4.6 PowerPC 24
2.4.7 x86 25

3 Large-page Policy **29**

3.1 Large-page policy approaches 29
3.2 Global 29
3.2.1 Fixed multiple page sizes 29
3.2.2 Pinning 30
3.2.3 HugeTLB 30
3.2.4 Winwood et al. 31
3.2.5 Solaris 32
3.3 Static 33
3.3.1 HP-UX 33
3.3.2 Shimizu and Takatori 34
3.3.3 Fragmentation Issues 35
3.3.4 Buddy Allocators 35
3.3.5 Slab Allocators 36
3.4 Dynamic 36
3.4.1 IRIX 36

3.4.2 Promotion . . . 37
3.4.3 Reservation . . . 38
3.4.4 Contiguity Daemons . . . 42

4 Comparison Summary . . . **45**

5 Research Questions and Conclusions . . . **47**

5.1 Research Directions . . . 48

Bibliography . . . **49**

1 Motivation

1.1 Virtual Memory and the TLB

Virtual memory underpins the operation of almost all modern general-purpose computing. By providing the abstraction of an address space, the programmer is released from many difficult and time consuming tasks related to memory management, and the operating system is free manage underlying memory as it sees fit.

Rather than managing virtual memory on a byte-by-byte basis, the system defines the smallest unit of memory as a *page*. This is in the order of some kilobytes generally within a range of 4KiB – 64KiB.

A virtual address must be translated to a physical address for hardware access. As this is a common operation, it benefits greatly from a cache of these translations; by convention we call this cache the *translation lookaside buffer* (TLB).

TLB *coverage* refers to how much virtual-memory address space can be translated by the limited number of TLB entries; more is universally better. In a modern system with gigabytes (or more) of physical memory running many large applications, overheads from TLB misses and consequent refill costs can easily become a bottleneck to system performance. This problem shows no signs of abating.

1.2 Increasing TLB coverage

There are essentially two ways to increase TLB coverage [TKHP92] :

1. more TLB entries
2. make each entry map a larger page

1.2.1 More TLB entries

Unfortunately, the TLB does not scale upwards easily. A traditional TLB design uses *content-addressable memory* (CAM) for high-speed parallel lookups. Due to its design, CAM does not scale up well in terms of transistor usage (and hence die space), power usage or speed.

One method used by hardware designers to scale up cache size is to divide the cache into "buckets" of entries, and search each bucket in parallel for a match. This is termed a *set-associative cache*. A TLB can be implemented with a set-associative cache, however it raises considerable problems if the TLB is to support multiple page sizes. At this point, it is sufficient to note that all modern processors have multiple page size support, and thus set-associativity is not a good solution. We examine the issue fully in Section 2.1.1.

Another concern is the interaction with the main processor cache. A physically-tagged cache requires TLB translation of virtual addresses before the cache lookup can complete [Wig03]. Hence a physically-addressed L1 processor cache's performance is directly tied to the speed at which the TLB can complete a translation. If a larger TLB is slower, it will have a flow-through effect to the overall memory latencies of the system and hence overall performance.

Virtually-addressed caches avoid the problems of requiring a TLB lookup before finding data, since a match is made on the untranslated virtual address. However, since one physical frame may have many virtual addresses *aliased* to it, keeping consistency within the cache becomes much harder. These aliasing issues are the primary reason virtually-addressed caches are not more popular in modern hardware implementations.

Some alternative schemes, such as the Itanium pre-validated cache design [BMS02, Lyo05], can help improve TLB and cache interaction. However, even with alternative TLB addressing schemes, the small size of pages relative to the *working set* [Den68] of a modern computing processes (kilobytes compared to gigabytes or even terabytes) means anything other than an extraordinary increase in TLB size will still leave TLB space at a premium.

1.2.2 Larger Pages

If we cannot easily scale up the number of entries, we could consider increasing TLB coverage by increasing the range of addresses entry translates. This equates to having a larger page size.

A larger page size will naturally decrease the overheads of managing pages (since for the same amount of memory, there are few pages to manage) and increase TLB coverage. However, there are several critical disadvantages to a blanket increase of base page size, which we examine below.

1.2.2.1 Fragmentation

Some of the trade-offs of increased page size have been evident since the first virtual memory implementations:

> There is a page size optimal in the sense that storage losses are minimized. As the page size increases, so increases the likelihood of waste within a segment's last page. As the page size decreases, so increases the size of a segment's page table. Somewhere in between the extremes of too large and too small is a page size that minimizes the total space lost both to internal fragmentation and to table fragmentation. (Denning, 1970 [Den70])

In the quote above, Denning is referring to the concepts of *fragmentation*.

If a page of memory is not fully utilised because the object it is storing is smaller than the page size, we refer to the left over, unusable space as *internal fragmentation*.

If we reduce the page size we reduce internal fragmentation, but our allocations become more scattered in memory, with possibly many small holes between the allocations. *Contiguous* memory refers to a consecutive array of physical memory larger than the page size. This is often either required or useful; for example I/O devices doing direct memory access (DMA) may require contiguous memory, or the increased locality may provide for increased performance of an application. Many small gaps are referred to as *external fragmentation* and inhibit contiguous allocations.

Fragmentation has long been studied in the literature. Wilson et al. [WJNB95] identify fragmentation in general as "an inability to reuse memory which is free". They further identify that it is very difficult to quantify algorithmic approaches to reducing fragmentation. Consider that the behaviour of a memory allocator algorithm depends on three elements:

1. The size of holes available for allocation.
2. The size and lifetime of future requests.
3. The behaviour of the allocator.

Each of these elements interacts with the other; for example the behaviour regulates which holes are free for future allocations, which depending on object lifespans can be either positive or negative (e.g. if the allocator leaves small holes, and all future requests are for small objects that fit in the holes, it is more successful).

Wilson el al. identify the root cause of fragmentation as placing objects with dissimilar lifespans in adjacent areas. If object lifespans and allocations were completely random it would make it impossible to create an effective allocation scheme to avoid fragmentation. Observations, however, show more regularity in program behaviour.

They identify several common classes of memory behaviour in programs:

- **Plateaus** are programs which allocate a large amount of memory, but use the data for a long period.

- **Ramps** are programs where memory allocation grows slowly over time, without intervening freeing of memory. Both ramps and plateaus profiles reduce the need for the allocator to reuse freed memory, but small holes between large, static allocations can cause problems.
- **Peaks** are programs which build up a large object, use it for some time and then discard it. They do this several times, for each individual phase of the program. Clearly freeing of memory is an important consideration here; any small survivors of the freed peak may interfere with further allocations.

This nomenclature will be useful when describing techniques in later sections, as each has different implications for memory fragmentation.

1.2.2.2 Architectural Limits

The page size is an architectural limit of the processor. The operating system can only choose those page sizes which the processor supports, which may be limited.

Insufficient flexibility in page size choice may lead to suboptimal behaviour and significant problems with fragmentation.

1.2.2.3 Page size choice

The optimal page size will vary between applications running in the system; one size does *not* fit all. For example, implementations on IRIX [GS98] found some scientific load benchmarks which show a pronounced "sweet-spot" page size. After this point, increasing the page size further meant overheads for allocating sufficient aligned memory began to outweigh any advantages gained.

Caşcaval et al. [CDSW05] found similar results with K42 on PowerPC hardware, where larger pages and consequent differing alignments caused adverse effects from cache interference and memory bank contention; one application showed a 39% slowdown with larger pages.

1.2.2.4 Operating System Concerns

Larger pages can effect performance of any I/O operations, since the smallest amount of data the system deals with is generally the page size. Larger transfer sizes have a trade-off between increased latency and increased bandwidth. This can influence the performance of critical OS components such as memory mapping of files and swapping.

More subtle issues also appear in other subsystems of the operating system. One of the most important performance improvements provided by a modern operating system is the *page cache* where on-disk data is kept in unused memory to speed access.

If the base page size is increased this leads to more internal fragmentation (and hence wasted space) within the page cache due to the "left-over" data on the last page of the file. Kleikamp and Pulavarty [KP06] suggest a method of coalescing these small *file tails* of multiple files in the page cache of Linux into a single managed area. Results of the work are not available in the paper, but the approach looks interesting.

1.2.2.5 Other Roles of the TLB

The TLB traditionally keeps and checks permission attributes for each of its translations (e.g. ability to read and/or write). This is used to enforce protection within the system, ensuring a process only utilises memory correctly allocated to it.

Since the TLB operates only at the page size, larger pages thus mean the *protection granularity* is increased. Increased granularity is undesirable since it promotes either wastage (via needing separate larger pages for different shared objects) or unsafe sharing. Schemes such as a separate protection cache [WWTH03] which allows de-coupling of protection from the TLB (and hence page size) have been proposed, but are not implemented in current hardware.

1.3 Multiple Page Sizes

If we are to accept that TLB sizes are not increasing and that increasing base page size is not without significant disadvantages, we are left to consider the possibility of multiple page sizes within the one system.

1.3.1 Superpages defined

We will term a contiguous range of virtual addresses a *mapping*; this should be thought of as representing some arbitrary sized object. A mapping may grow or shrink — for example a mapping representing a file grows as it is written to.

A mapping is *backed* by physical frames of memory. In the traditional case we have considered until now, the mapping is divided only into base page size regions, each of which maps to a unique physical frame. With superpages the mapping is divided into irregular sized regions; each region is at least a base page size large but could be some power-of-two multiple of this, as implemented by the hardware[1].

When this larger region is backed by physically contiguous memory, we can use a single, larger, TLB entry for it. We call any region larger than a base page size mapped with a single, larger TLB entry a *superpage*.

Judicious use of superpages can make more effective use of existing TLB resources without the drawbacks of a simple page size increase.

1.4 Overview

How to support multiple page sizes effectively is the focus of rest of this paper.

In Section 2.1 we examine the issues that multiple page sizes raise for traditional TLB designs. In Section 2.2 we examine approaches to avoiding these problems from the literature. We then provide a short survey of multiple page size support in current commercial processors in Section 2.4.

Memory policy is under the control of the operating system, and is the major focus of Chapter 3. We firstly categorise operating systems approaches to supporting large-pages in Section 3.1, and complete the chapter with analysis of literature and existing implementations in this framework.

We conclude with a presentation of open research questions and challenges in Chapter 5.

[1]The number of pages which make up a superpage is always a power of 2 integer, e.g. 2, 4, 8, 16 etc. This is because dedicating another bit of a virtual address to offset doubles the size of the offset.

2 Hardware support for multiple page sizes

Chapter 1 outlined the motivation for multiple page size support. Below, we firstly examine some of the constraints to multiple page size support, secondly examine existing research to overcome these constraints, and finally examine the features of existing architectures with respect to multiple page sizes.

2.1 Hardware constraints

In Section 1.2.1 we identified a set-associativity as an inhibitor to multiple page size support. Below we examine this claim in more detail.

2.1.1 Set-associativity

In a single-page-size system, any virtual address presented to the MMU can be unambiguously split into a *virtual page number* (VPN) and offset. The VPN is presented to the TLB, which will consequently provide the underlying physical page.

When multiple page sizes are used, a given virtual address no longer uniquely identifies a virtual page number. The split between VPN and offset bits will depend on what size page the given address is currently mapped as [TKHP92, Sez04].

In a fully-associative TLB, the VPN of each entry is checked for a match individually, in parallel. Since each entry is checked individually, each entry can be extended with a mask field to implement multiple page sizes, as illustrated in Figure 2.1.

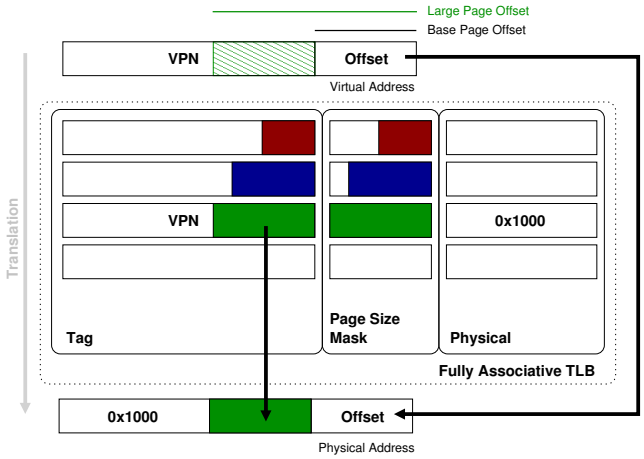


Figure 2.1: A fully associative TLB can be easily extended for multiple page size support by adding a mask field, which sets the page size (and hence offset bits added to the physical address) for each entry. Larger pages have more bits of the mask set, whilst a base page size has no bits set.

A fully associative TLB is expensive to create in hardware and thus limited in size. Thus a larger TLB is usually implemented via *set-associativity*.

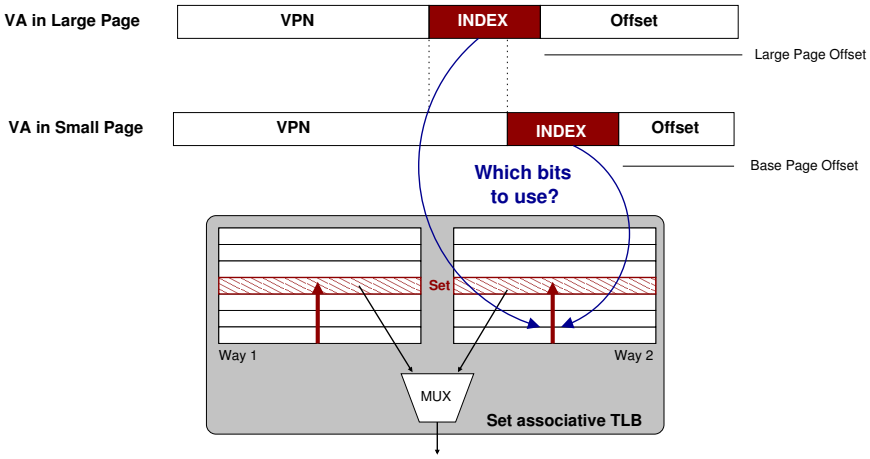


Figure 2.2: An illustration of issues arising from supporting multiple page sizes in a set associative TLB. A set associative TLB separates the TLB into *ways*; an index is taken from the virtual address, and the entry at this index in each way is checked simultaneously for a match. The TLB must know the index before it starts the process, but we can't know this until we know the page size, which is kept in the TLB!

Set-associativity separates the TLB into several *ways*, which each hold a portion of the TLB entries. At translation time, a number of bits are used to index into each way; the entry at this index in each way is then checked in parallel for a match. Thus the parallel component of the lookup is restricted to the number of ways, rather than the total number of entries as in a fully-associative cache. We see an illustration of this in Figure 2.2.

The index into the way must be known before the lookup can start. However, when presented with only a virtual address the TLB has no information to distinguish the page size of the given virtual address, and hence no way to find the split between offset and index bits.

It is a classic chicken-egg problem; we need the page size to index the TLB, but the page size is kept in the TLB!

2.1.2 Discussion

There are a number of potential solutions, which we will discuss below.

If one were to use the "worst case" solution of always assuming the largest page size, then for every n bits of overlap 2^n small pages will compete for the same TLB set. This competition for space causes small pages to have a much higher *conflict miss* penalties.

Should we optimise for small pages and use the overlap bits as index bits in all cases, we effectively negate the advantages of saving TLB entries that large-page support brings. Consider that since offset bits are used as index bits, there is no savings in the number of index entries required.

Another approach is to increase set-associativity (i.e. the number of ways) such that each smaller page can find an entry in a set even when indexed as a larger page. This may be practical for small, limited page sizes (for example, supporting 4KiB and 32KiB means an 8-way associative cache) but becomes impractical when dealing with many varying page sizes.

In a similar vein, a form of sequential access where different indexes are checked in order could be instituted. This means effectively turning a single lookup into as many lookups as there are page sizes; too great a penalty for the speed-critical TLB. This could possibly be avoided by adding more ports to the TLB, but electrical considerations generally make this impractical, and again it does not scale to many page sizes.

Another solution is to use distinct TLBs for different page sizes [Sez04]. The disadvantage of this approach is two fold. Firstly, by partitioning the TLB no one page size can populate all the available space, leading to wastage. Secondly, the scheme does not scale up with more page sizes. More TLBs increases power and space

requirements, but perhaps more importantly raises complexities in ensuring consistency, especially important on multi-processor systems.

2.2 Hardware-Based Approach

The problem of multiple page sizes in set-associative caches has been the focus of much research. We examine a range of these approaches below.

2.2.1 Sub-blocking

TLB sub-blocking is a technique inspired by sub-block cache designs.

2.2.1.1 Sub-blocking cache

For a given cache size, a smaller line size means more tag bits required to find the index for a given address. Smaller line size can reduce miss penalties as less data needs to be brought into the cache to satisfy the miss. However larger tag sizes introduce hardware constraints (power, transistor count, layout, etc) and also decreases the ratio of metadata (tags) to payload in the cache.

Sub-blocking caches were originally created to avoid this trade-off between expensive parallel tag matching hardware and large line sizes in early processors [HS84].

A *sub-block cache* uses a larger line size, but divides a line into two or more *sub-blocks* each with their own valid bit. A tag match now requires an additional step of checking that the given cache word is present.

2.2.1.2 Sub-blocking TLB

Talluri and Hill [TH94] propose *TLB sub-blocking* as a method for improving superpage support with minimal operating systems changes.

They divide sub-blocking up into two categories:

- Complete sub-blocking
- Partial sub-blocking

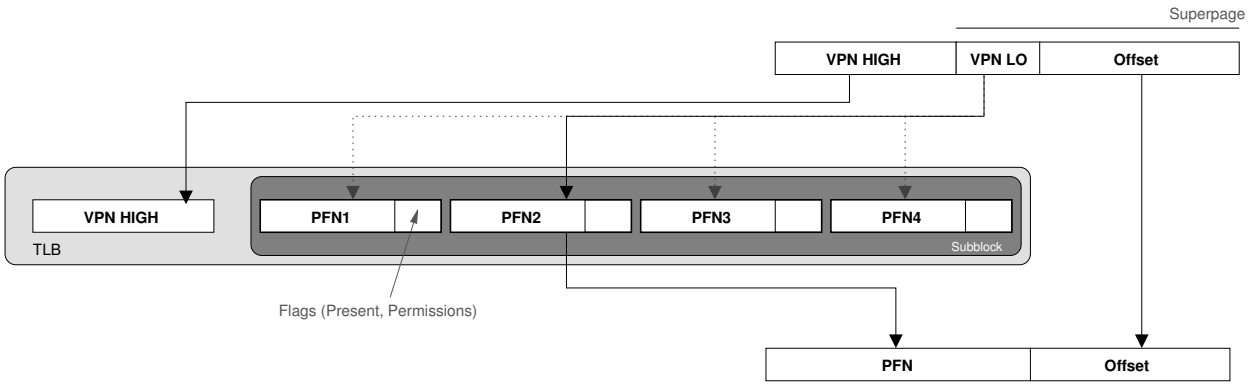


Figure 2.3: Complete sub-block TLB [TH94]

A complete sub-block TLB, illustrated in Figure 2.3, keeps a separate physical frame number (PFN) for a number of sub pages (the *sub-blocking factor*). This clearly allows for arbitrary and very transparent support of superpages, since a contiguous virtual-address range can be mapped onto non-contiguous underlying physical frames.

The complete sub-block TLB requires a very wide, parallel accessible RAM array, which is expensive in both chip real-estate and power requirements. Additional overheads from multiplexing multiple path choices and additional control logic can also slow the TLB [Tal95].

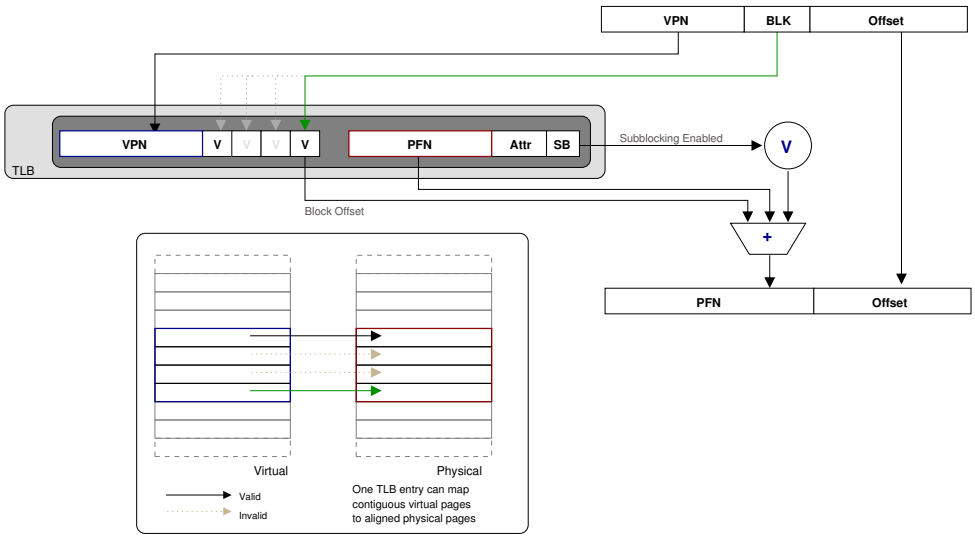


Figure 2.4: Partial sub-block TLB [TH94]. Note that the virtual address has a block offset, which corresponds to the *valid* bits stored with the VPN. Thus one TLB entry can map contiguous virtual addresses to aligned physical frames.

As an alternative Talluri suggests the *partial sub-block TLB* [Tal95], illustrated in Figure 2.4. Rather than keeping a unique physical translation for each subpage mapped by the TLB entry, only a single translation is kept and used as a base to offset into an aligned group of pages.

Talluri's scheme handles virtually contiguous but physically un-contiguous pages by replicating the VPN in a separate TLB entry. This creates an *synonym* problem similar to the aliasing issues with a virtually addressed cache. Although two entries in the TLB can have the same VPN, if the subpage valid bits are mutually exclusive between entries with the same VPN they may be considered part of the tag.

Another problem is alignment between virtual addresses and physical frames. If the PFN field holds only enough bits to map the physical address space in sub-block regions, a given block offset in the VPN block must point to the same block offset in the physical sub-block. To be more concrete, the `BLK` field in Figure 2.4 is untranslated, so if it refers to sub-block `2` in the virtual page, that virtual sub-block must map to physical sub-block `2` in the physical page. Talluri avoids this by introducing a sub-blocking flag (illustrated) which turns sub-blocking on or off for the entry. If sub-blocking is off, then the `BLK` field is ignored and the entire VPN is translated.

The penalty is that the physical address bits stored are increased to be able to reference an arbitrary physical page. Any unaligned mappings will take up an entire TLB entry and so lose the potential to store other sub-blocks. These effects can be somewhat mitigated by the operating system ensuring suitable layout.

2.2.1.3 Discussion

The advantage of a sub-blocking TLB is that data area, rather than expensive tag area, is replicated. This allows the sub-block TLB to maintain much larger coverage without the increased costs of a larger single page-sized TLB.

A sub-blocking TLB can naturally support multiple page sizes. In the worst-case scenario, data is badly aligned, meaning replication of entries, possibly to the point of every page requiring its own translation. Since this equates to a traditional TLB design, we estimate the overheads of the sub-blocking TLB would, for the extreme worse case, be a disadvantage. However, the operating system can mitigate this by ensuring allocations happen in a TLB friendly fashion.

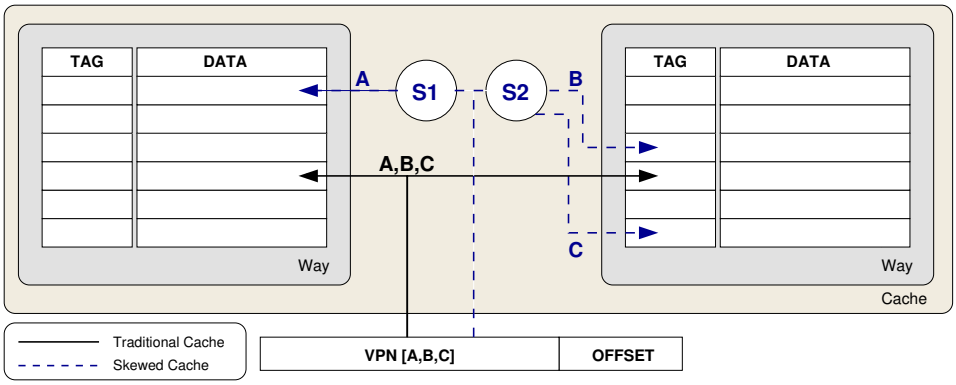


Figure 2.5: Skewed Associative Cache. `S` represents a hashing or skewing function for each way. Note that with a traditional cache, there is a conflict miss for addresses `A,B,C`, whilst the skewed associative cache design distributes the addresses by hashing them to different locations. [Sez93]

2.2.1.4 Shadow Memory

A related method for implementing superpages is presented by Swanson et al. [SSC98]. A *memory controller TLB* (MTLB) is utilised to map unused portions of the physical address space to physical memory (for example, the extra 3GiB in a 32-bit system with only 1GiB of installed RAM).

The MTLB intercepts addresses on the memory bus, and thus can make physically dis-contiguous frames appear contiguous. As with a sub-blocking TLB, this allows for transparent large-page support. This scheme, however, moves the translation information from the expensive TLB space to the memory controller.

The scheme showed significant performance increases of between 5–20% over general purpose workloads. There are some limitations; since the memory controller re-maps unused physical space the advantages diminish as RAM is added, penalising the higher-end machines (although most modern servers have a much larger than 32-bit physical address space). The re-mapping idea was implemented in the Impluse [ZFP+01] memory controller, but is not currently implemented in any commercial architectures.

2.2.2 Skewed TLB

A *skewed TLB* [Sez04] design comes from similar work with a skewed cache design [Sez93].

A skewed cache attempts to reduce conflict misses by mapping the same virtual address to different sets in a multi-way cache. We illustrate this in Figure 2.5; for addresses `A,B,C` which map to the same set, the traditional cache design must resolve a conflict miss. Seznec suggests that each way of the cache have a unique *skewing function* (S) which modifies the index for each virtual address.

The advantage of a traditional set associative cache is that each virtual page is no longer only able to be found a particular set. Since each virtual address has a unique home, each entry can be extended with a page size mask as described previously for a fully associative cache.

Seznec describes mechanism and a proposed implementation, but does not describe results. Currently, the scheme is not implemented in any commodity hardware.

2.2.3 Zip Code TLB

A *zip code TLB* [Lie96] attempts to remove contiguity of indexing by adding an element of randomness to the TLB index. The top bits of a virtual address are reserved as the *zip code*; this can be hashed with the lower virtual page number to implement an index.

The choice of zip code is an effective way of colouring the TLB, such that entries do not compete. As can be seen in Figure 2.6 some bits of the incoming virtual address are hashed with the incoming virtual page

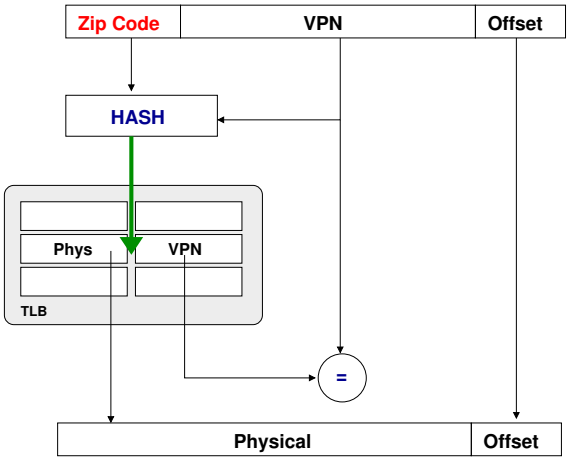


Figure 2.6: Liedtke Zip Code TLB [Lie96]

number to create a unique TLB index. Since the zip code is not related to the page size, the scheme is page size independent.

Disadvantages of this scheme are that the zip code decreases the address space bits available since they are utilised for indexing the TLB.

By default, any number of zip code–virtual page number pairs could map to the same entry, necessitating an entire TLB flush to purge an entry. Liedtke proposes a number of schemes to avoid aliasing problems. The common element is that translations are checked via the page tables to avoid aliases (XXX details of the four schemes?)

2.3 Software Approaches

2.3.1 Software-managed address translation

One potential solution to removing problems related to superpage mappings and the TLB is to remove the TLB entirely! Such a scheme has been referred to as *software-managed address translation* [JM97] or *in-cache address translation* [WEG+86].

Architectures pass varying levels of control over memory management unit (MMU) operations to software. In the extreme case, processors such as the Intel IA32 processors [Int01] and ARM processor [Int99] enforce a particular page table structure and walk it in hardware. The Intel Itanium processor has a more flexible hardware walker that can walk over a virtually or physically linear page table [Int00]. Other processors such as MIPS64 [MCY97] and UltraSPARC [Sun05a] implement a software loaded TLB[1], encouraging small, fast fault handlers with pre-computed offsets.

However, since the processor operates on data gathered from the cache, if the cache is virtually-indexed there is no need for a separate TLB. Any time there is a request for a virtual address, the following series of steps happens:

1. The cache is directly checked for the line containing that data. Should it be available, processing continues. Else

2. A virtually-linear array is indexed and searched for a translation entry. Should this translation be available in the cache, the translation is de-referenced, the physical data line fetched into the cache and processing continues. Else

[1] Alpha [Com99] has a software loaded TLB, but the implementation is fixed in firmware code, so appears fixed to the operating system layer.

3. A fault is raised to the operating system, which can load the appropriate data.

Since control is handed back to the operating system, it can implement any page table mechanism it likes. The virtual address is never translated by the hardware, so there are no problems in supporting multiple page sizes.

Wood et al. [WEG+86] showed that in comparison with small TLB sizes an in-cache address translation scheme is viable. Jacob and Mudge [JM97] show that overheads for a more modern superscalar design can be reduced to between 0.05 and 0.02 cycles-per-instruction (CPI), depending on the behaviour of the operating system.

However, since the software managed translation relies on virtually-addressed caches, implementations must deal with the problems virtual caches introduce.

Since the cache is passed a virtual address, there is clearly the potential that two virtual addresses may actually refer to the same underlying physical page. When two virtual addresses refer to the same physical page we say that they are *synonyms* (in language, a word having the same meaning as another word) or are *aliases* for a physical page.

Synonym-related problems have been dealt with in many ways — for a review see Wiggins [Wig03] — but the simplest scheme is to introduce a global address space such that shared data always appears at the same virtual addresses.

Protection (including accessed and dirty bits) is usually handled by the TLB; alternative schemes either bring protection bits into the cache line or have a separate TLB like structure exclusively for protection information (a *protection look aside buffer*).

Specifically, in some implementations the dirty and reference are stored and updated by the TLB. Since the PTE may not be accessed on a cache write (per step 1 in the above sequence) there is no easy way to set a referenced bit (used for implementing any LRU type schemes). This can be approximated by setting a *missed* bit on the PTE in step 2 above. Dirty bits can be handled similarly, but flushing dirty lines now requires lookups, an expensive process. To avoid this, the physical address can be kept as part of the cache data, to facilitate fast write-back.

However, this raises yet further problems. Tzou [Tzo89] identified that problems with in-cache address translation are fundamentally the same as multi-processor TLB consistency problems.

Consider the three places that translation information is now stored

1. The underlying PTE entry in the page tables (main memory)
2. The cached copy of the PTE
3. A physical translation in the cache line (for fast writeback)

The regular cache-coherency mechanism takes care of synchronisation of items 1 and 2. However, since updating a PTE is the equivalent of a memory write, there is no easy way to update item 3. Hardware would need to intercept the update, find the page it referenced and flush it from the cache of all processors in the system.

The significant problems associated with software-managed address translation have thus kept it from gaining significant commercial implementations.

2.4 Multiple Page-Size Support in current processors

An overview comparison of MMU-supported page sizes for common processors can be seen in Table 2.1

2.4.1 Alpha Processor

The Alpha architecture requires a software *Privileged Architecture Library Code* (PALcode) as an intermediate software layer for handling (amongst other things) loading of the MMU. PALcode runs in a privileged mode with interrupts disabled, so its operation appears atomic to the operating system.

PALcode allows for simplified hardware implementations, since microcode is not included on the chip its self. The re-loadable nature provides for greater flexibility of implementations; most operating systems use a standardised version, however examples such as L4/Alpha have run microkernel services inside PALcode [Pot99].

PALcode is responsible for loading translations into the *translation buffer*. The architecture defines granularity hint bits within a translation entry which flag the entry as mapping either 8, 64 or 512 contiguous 8K pages. The architecture may choose to implement or ignore these hints, so all possible translation entries at the base page size must be marked with the same size hint, else undefined behaviour occurs.

The Alpha implements multiple page sizes with a single fully associative TLB.

2.4.2 MIPS R10000

The MIPS R10000 [MCY97] has a 64-entry fully-associative TLB with 7 different page sizes; 4KiB, 16KiB, 64KiB, 256KiB, 1MiB, 4MiB and 16MiB.

The TLB is implemented with 2-way sub-blockeding where each TLB entry maps *two* pages, as decided by the low bit of the VPN. MIPS implements a fully software loaded TLB, so all policy is under the control of the operating system.

To support multiple page sizes, a *PageMask* register is provided which selects the page size when doing a TLB read or write. For an operating system with no large-page support this can be set as a default, however if large-page support is needed the software fault handler will need to be checked with different page size masks to find the correct entry.

2.4.3 SPARC Processor

The Scalable Processor Architecture (SPARC) has a long and interesting history. The SPARC processor grew from work on both the original Berkeley University RISC architecture (1980-1984) and the Stanford University MIPS architecture (1981-1984), both of which emphasised what is now known generally as a Reduced Instruction Set Architecture (RISC). SPARC refers to an abstract processor design specification; implementors take the SPARC architecture specifications and implement it as hardware. The current SPARC specification is SPARC V9.

The SPARC architecture makes minimal MMU demands, leaving details up to implementors. This is to facilitate a wide range of applications from embedded (where no MMU may be appropriate) to server.

The only architecture requirements are for a range of *address-space identifiers* which are prepended to every address. The the ASIs have two roles; firstly as a unique tag on address spaces, reducing context switch overheads, and secondly to map internal processor resources (registers) to addresses.

A range of addresses in an ASI is a *context*; the only two required contexts are the *primary* and *secondary*, which hold their target ASI in a processor register (others, such as the *nucleus* are optional). A range of reserved ASIs provide different views for the primary and secondary contexts, such as big/little endian or no–fault semantics. Load and store instructions by default reference the primary context, alternate forms use the secondary context or can be given a particular ASI to use. Further reserved ASIs provide for access to processor and other system resources, whilst the upper range is left for system use. The relationships are illustrated in Figure 2.7.

2.4.3.1 UltraSPARC

Since the MMU is implementation specific, we examine the most common implementation of SPARC, Sun's UltraSPARC. The major UltraSPARC product lines are listed below.

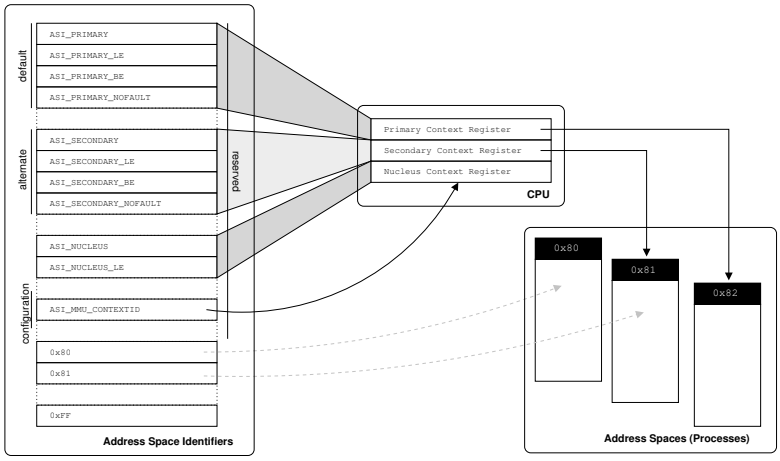


Figure 2.7: SPARC processors define a number of address-space identifiers. The primary and secondary contexts, identified by a register value, have a number of associated addresses spaces with different properties, such as endianess and caching policy. Other address spaces provide for access to configuration or register values. The highest addresses spaces are left for processes running in the system.

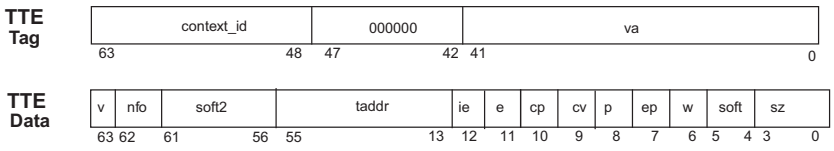


Figure 2.8: A `sun4v` *translation table entry* [Sun05a]

Name	Description
`sun4c`	SPARCStation IPC
`sun4m`	Classic, SPARCStation 5/10
`sun4u`	UltraSPARC
`sun4v`	Niagara (chip multi-threading and hypervisor)

With the SPARC V9 growing older, Sun released an updated *UltraSPARC Architecture 2005 Specification* [Sun05a] which is superset of the SPARC V9 architecture and many additional extensions. It fully supports a hypervisor layer, and documents MMU characteristics. The first implementation of this revised architecture is the Sun UltraSPARC T1, commonly referred to as "Niagara".

The UltraSPARC TLB is referred to as a *translation table*. A *translation table entry* (TTE) consists of the context, a virtual address, the matching physical address and a number of attributes, as illustrated in Figure 2.8.

Figure 2.8 refers to a `sun4v` TTE entry, which is slightly different to the older `sun4u` format. Of particular interest is the page size (`sz`) field, which is now specified as 4 bits rather than the 3 allocated in the older format. Current hardware, however, does not implement all bits.

UltraSPARC defines a software-loaded TLB, so all faults are resolved directly by the operating system handlers (UltraSPARC and MIPS are the only modern processors to maintain a software filled TLB). To facilitate quicker loading of TTE entries, UltraSPARC provides some hardware support for a *Translation Storage Buffer* (TSB).

A TSB is a linear array of TTE entries kept by the operating system in main memory as a cache of the underlying page tables (also referred to as a software-TLB). On a TLB miss, the processor will pre-compute an offset into the current TSB. The TTE entry at this offset can then be quickly checked and loaded if appropriate, saving the overheads of a page table walk.

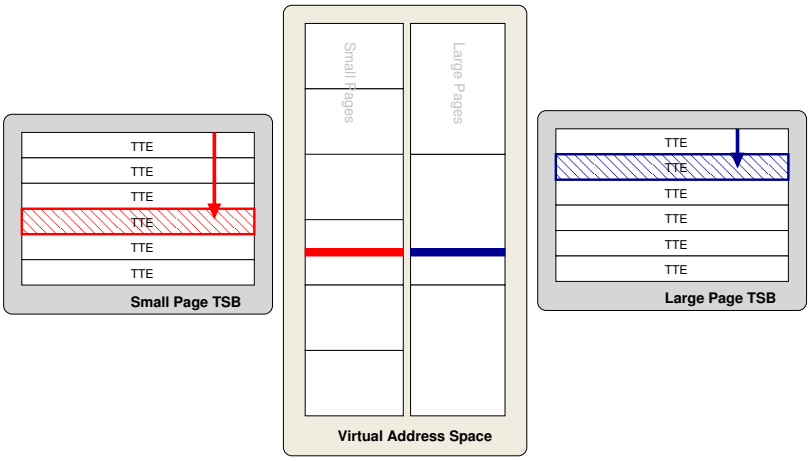


Figure 2.9: SPARC processors pre-calculate an offset into a cache of translation entries (TSB). It does this for two user specified page sizes, but since fault handling is under software control alternative offsets can be quickly calculated.

Multiple page sizes confuse the situation, however. As illustrated in Figure 2.9, for the same faulting address the TSB offset will differ depending on the page sizes used. To aid with this, the processor calculates *two* offsets for the system based on user specified page sizes[2]. The software fault handler can then choose the correct offset based on its knowledge of the fault addresses. If more page sizes are required the software can manually calculate offsets, taking the (small) penalty of extra time required in fault handlers.

Effective TSB management policy can have a large effect on system performance. One method of greatly increasing TLB coverage on the SPARC processor is dynamic creation and sizing of TSBs.

For example, Solaris originally statically allocated a fixed number of TSBs based on system memory at boot. Thus the operating system would often not have resources in the pool to allocate a suitable TSB to a process. Further compounding the problem was TSBs in the pool were a fixed size of either 128KiB or 512KiB, which left little flexibility since these sizes tended to be either too small or too big, and rarely "just right". The result is significant TSB sharing and hence contention in a busy system.

By allowing each process to have its own dynamically created (and dynamically sized) TSB significantly more TLB coverage can be obtained [MS02].

UltraSPARC solves the problem of aliasing by having multiple TLBs. For example, the UltraSPARC IIIc has three data TLBs, accessed in parallel. One small fully associative TLB can handle any page size, whilst two larger 512 entry, 2-way set associative TLB's can each be set to handle a single page size. This somewhat restricts arbitrary page size decisions, as page sizes that are not mapped by one of the two TLB's fall back and contend for space in the smaller fully associative TLB. On the newer UltraSPARC T1 ("Niagara") processor there is only a small fully-associative TLB.

2.4.4 ARM Processor

The ARM range of processors is primarily designed for embedded platforms, but supports multiple page sizes. ARM defines a hardware page-table walker, which necessarily needs to deal with multiple page sizes.

As illustrated in Figure 2.10 there is a two level page table hierarchy. At the 4096-entry first level, each entry is in one of four states:

1. **Invalid**
2. **Page table pointer** points to a further *leaf* page table entry

[2] On `sun4v`; older `sun4u` processors did this only for fixed 8KiB and 64KiB page sizes

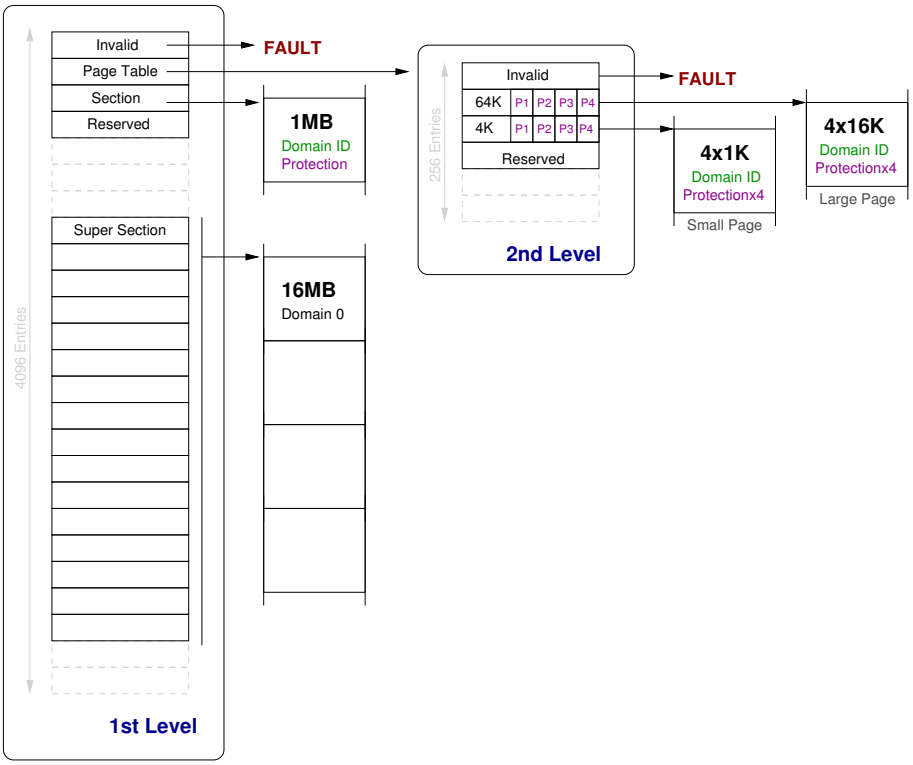


Figure 2.10: ARMv6 virtual address translation

3. **Section Pointer** points to an aligned 1MiB region of memory. This is given standard protection information, part of which is a *domain id* which is usually used by the OS as an address space identifier.

4. **Reserved** means reserved for future expansion

5. **Super Section** points to an aligned 16MiB region of memory. This takes 16 entries in the level one table, which need to be replicated, but will be mapped by the TLB with a single entry.

The second level page tables are similar, but each valid entry can map either a 64KiB or 4KiB region. Whilst they are tagged with the same domain, validity information is kept for four subpages (e.g. of size 16Kib and 1KiB respectively[3]).

Thus the ARM processor can support 4 page sizes in the TLB (16MiB, 1MiB, 64KiB and 4KiB) but enforces a smaller protection granularity for leaf pages.

Older StrongARM processors supported this model via a fully associative TLB [WH00]. More recent ARM11 processors have a two-level TLB. The first level is a split (instruction/data) fully associative *microTLB* of 10 entries, providing a translation in a single cycle. A larger *Main TLB* is further split into two parts; a smaller 8-entry fully-associative TLB which supports pinning of entries, and a larger 64-entry 2-way set associative component.

(XXX: how does it do it?)

2.4.5 Itanium

Itanium has a very flexible MMU with many interesting features aimed at improving translation performance.

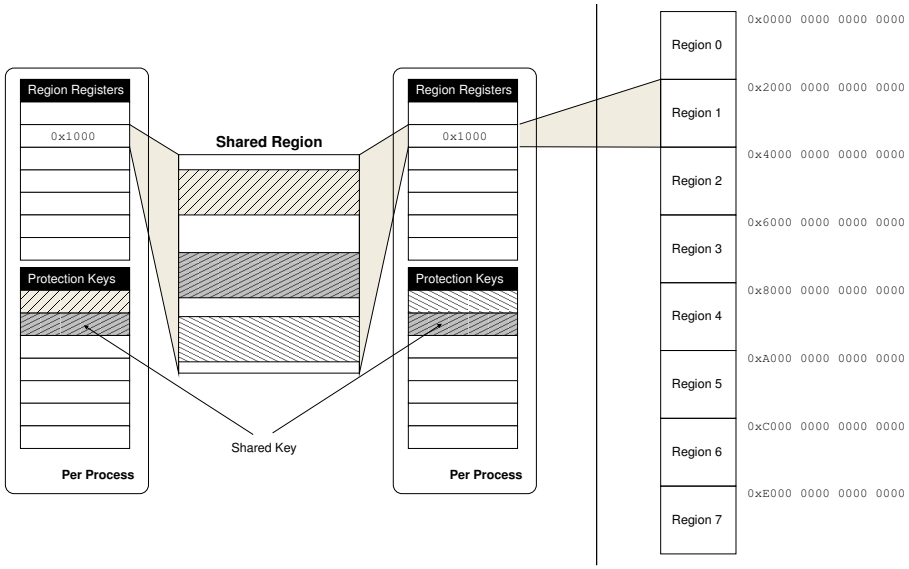


Figure 2.11: Itanium regions and protection keys. By giving both processes the same region ID, they have the same view of that portion of the address space. Protection keys allow even finer grained sharing, above each process has a private mapping and they share a key for another.

2.4.5.1 TLB Sharing

The goal of the TLB is to enforce unique views of the virtual address space for each process in the system. In the simplest scheme, when a *context switch* activates a new process, and hence virtual address space, the TLB is emptied or *flushed* to avoid the same virtual address in different addresses spaces mapping to the same physical address. This ensures the new process never sees old translations, and that translations for the newly activated virtual address space are reloaded by the operating system, which can implement policy and security. This is, however, a significant penalty since reloading from the operating system page tables on each context switch is an expensive operation.

Thus a common enhancement is to tag each TLB entry with an *address space ID* (ASID). This ensures only translations for the current active address space are matched without requiring emptying of the TLB.

Having a single tag for the entire address space reduces the ability of two address spaces to share TLB entries. We can have two execution contexts within a single address space; each execution context is termed a *thread*. The TLB is unable to enforce protection between threads, since they share the same ASID. However, threads share TLB entries, reducing overall TLB pressure through less duplication and increasing the apparent size of the TLB.

Itanium allows for the benefits of sharing of virtual address spaces at a much lower granularity than simply the entire address space. The Itanium divides its 2^{64} bit address space up into 8 *regions*, as illustrated in Figure 2.11.

Each process has eight *region registers* as part of its state, which hold a *region ID* for each of the eight regions of the process address space. If two processes share a region ID, then they have the same view of that region. Consequently the TLB entries can be shared, reducing the need to flush and reload entries on context switches.

To allow even finer grained sharing, each TLB entry on the Itanium is also tagged with a *protection key*. Each process has a number of *protection key registers* under operating-system control. When a series of pages (say, a shared library) is to be shared, they are tagged with a unique key and each process allowed to access it granted that key. The TLB will check the key in the translation entry when the page is referenced against the keys the process holds in its protection key registers, allowing the access if the key is present or otherwise rasing a fault to the operating system. The key can also enforce permissions; e.g. one process can write to the shared region

[3] 1KiB sub-pages are deprecated in current releases

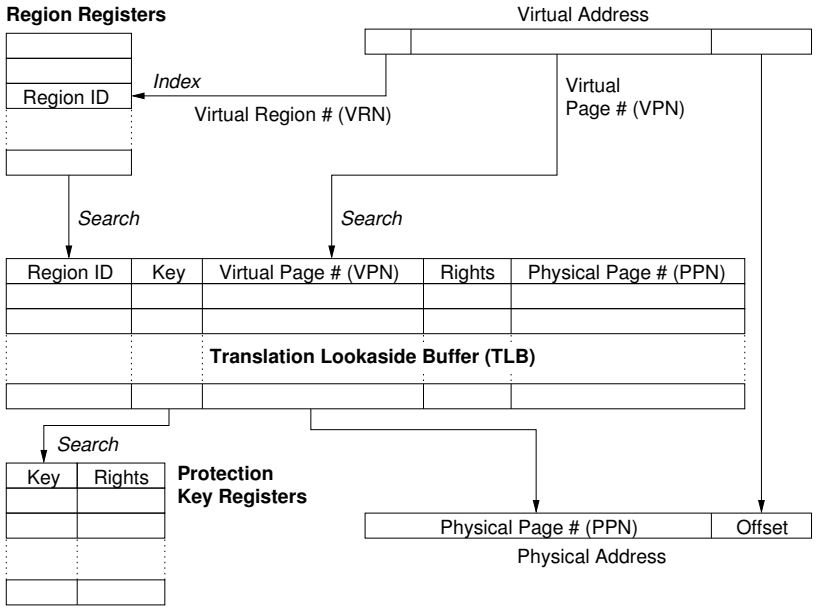


Figure 2.12: A view of the Itanium TLB translation process [GCC+05, ME02].

and another may have a read only key. This allows for more potential sharing of entries, and a consequent improvement in TLB performance.

An overall view of the Itanium translation process is provided in Figure 2.12.

2.4.5.2 Linear Page Table

At first, page-table structure may seem orthogonal to TLB performance issues. However, Itanium implements hardware loading of TLB translation in order to reduce the cost of a TLB miss. First we examine the underlying structure, then the Itanium hardware loading implementation.

2.4.5.2.1 Linear Page Table Introduction

A linear page table describes a contiguous table of translations for an address space. A linear page table facilitates an extremely fast best-case lookup, since the target is found by simply taking the virtual page number divided by the size of a translation entry as an offset from page table base.

Unfortunately a physically linear page table is impractical with a 64-bit address space, since every page must be accounted for, whether in use or not. Consider a 64-bit address space divided into (generous) 64KiB pages creates $\frac{2^{64}}{2^{16}} = 2^{52}$ pages to be managed; assuming each page requires an 8-byte translation entry a total of $\frac{2^{52}}{2^3} = 2^{49}$ or 512GiB of contiguous memory is required for the table.

The usual solution is a multi-level page table, where the bits comprising the virtual page number are used as indexes into level pointers. For the realistic case of a tightly-clustered and sparsely-filled address space, page table overhead is kept to around the minimum size required to manage only those virtual pages in use.

2.4.5.2.2 Virtual Linear Page Table

We can, however, use the large virtual address space to our advantage; even 512GiB is only 0.003% of the 16 exabytes of address space provided by a 64-bit system. Thus we can create a linear page table in the virtual address space, and use the TLB to map virtual pages holding translation entries to the physical pages where the translation entries reside.

In our example, the last 512GiB of the virtual address space are reserved by the processor as a *virtual linear page table* (VLPT). On a TLB miss, the hardware uses the virtual page number to offset from the VLPT base

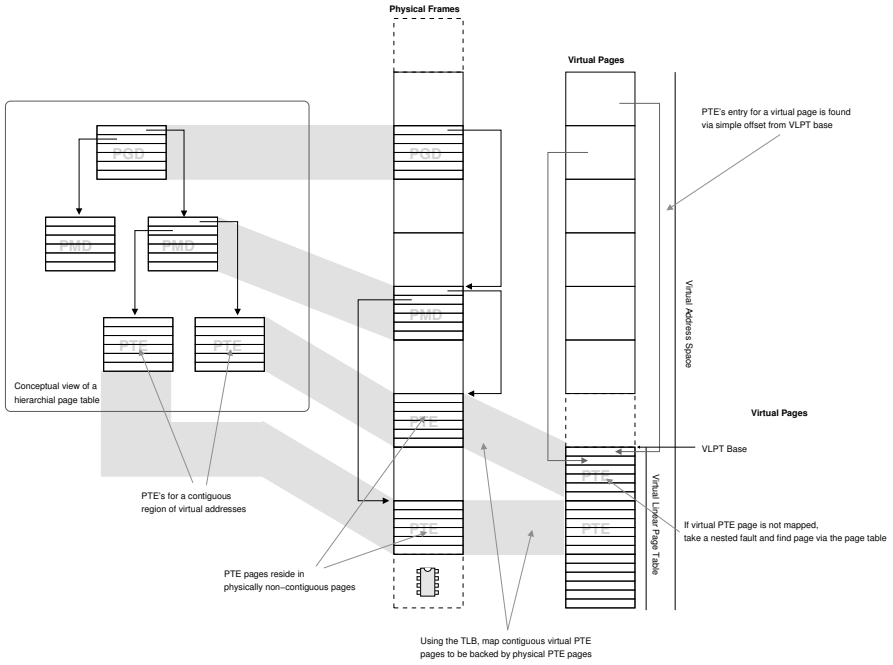


Figure 2.13: Itanium "short-format" virtual linear page table. The leaf entries of the operating system page table can be mapped into the virtually linear page table.

where it expects to find a suitable translation entry. If this entry is valid, the translation is read and inserted directly into the TLB.

However, since the translation entry in the VLPT is its self a virtual address, there is a possibility the virtual page which the translation resides in is not present in the TLB, and in this case a *nested fault* must be taken. At this point the page holding the translation entry must be found and mapped it into the VLPT; this is usually done by software.

We see the organisation utilised by Linux illustrated in Figure 2.13. Linux uses a multi-level page table within the operating system to keep track of virtual-physical translations of processes. When the nested fault is taken, the multi-level page table is walked to find the leaf page of translation entries in which the required translation resides; this leaf page is then mapped into the virtual-linear array. This works because a leaf page of a multi-level page table holds translation entries for a virtually contiguous region of addresses.

Once the virtual linear page table page is correctly mapped to a physical page holding translation entries the request can be re-tried; this time it will not raise a nested fault.

2.4.5.2.3 VHPT Hardware Walker

Itanium implements a VLPT in hardware, referred to as the *virtually hashed page table walker* (VHPT walker). On a TLB miss, the processor will calculate the offset into the VLPT and attempt to find the translation. If a valid translation is found, the hardware can directly insert the TLB entry and continue without raising an operating system fault; an invalid translation invokes the operating system fault handler. If the translation is not found, a nested fault is raised to the operating system which must insert a translation for the VLPT page mapping.

This has a number of consequences. Firstly, the advantage of the system comes when an application makes repeated or contiguous accesses to memory. Consider that for a walk of virtually-contiguous memory, the first fault will map a page full of translation entries into the virtual-linear page table. A subsequent access to the next virtual page will require the next translation entry to be loaded into the TLB, which is now available to the hardware walker and thus loaded very quickly, without invoking the operating system. We hope

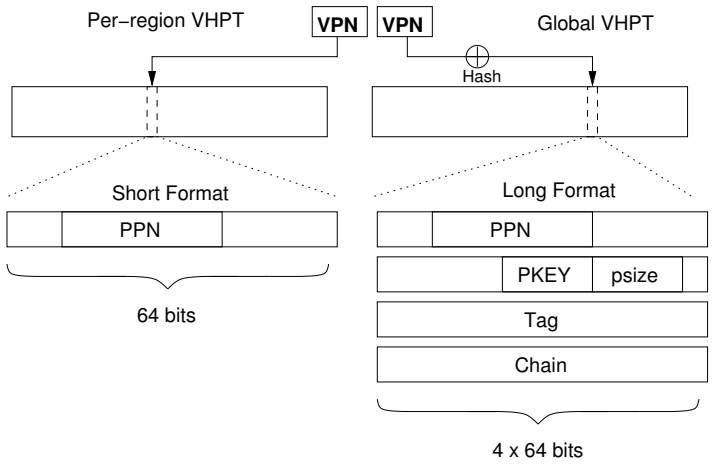


Figure 2.14: Itanium PTE entries [GCC+05]

to armortise the cost of the initial mapping over these faster accesses; a pathological case could skip over `page_size` ÷ `translation_size` entries and cause repeated nested faults, however.

Secondly, consider that the virtual page table now requires TLB entries, bringing an overall increase on TLB pressure. Again, we hope that higher capacity miss rates are regained in lower refill costs from the efficient hardware walker.

Thirdly, the hardware walker expects PTE entries in a specific format (the so-called *short format*; to be contrasted with the *long format* description below), and if the operating system is to use its page table as backing for the virtually linear page table without an intermediate layer (as in Figure 2.13) it must maintain this translation format.

Fourthly, there can be no efficient multiple page size support, since this would make the offset into the virtually linear page table no longer constant. To combat this, each of the 8 regions of the address space (Figure 2.11 has a separate VLPT which only maps addresses for that region. A default page size can be given for each region (indeed, with Linux HugeTLB, discussed in Section 3.2.3, one region is dedicated to larger pages).

We should note that we can in fact run with the hardware page-table walker turned off completely; in this case each TLB miss is raised directly to the operating system. The performance impact of this is considerable [CWH03].

2.4.5.3 Hashed Page Table

Using TLB entries in an effort to reduce TLB refill costs, as done with the VLPT, may or may not be an effective trade off. Itanium implements an alternative scheme with the potential to avoid these costs, but introduces an alternative set of trade-offs.

In this scheme, the processor *hashes* the incoming virtual address to find an offset into a contiguous *hashed page table*. Since there may be collisions, where two virtual addresses hash to the same entry, we must have a *tag* to distinguish the addresses. A *chain* is also included to point to another entry to try if the tag does not match.

The Itanium's alternative form of hardware page table walker is based on a hashed page table. Since each entry in a hash table requires more information, this is termed the *long format* virtual hashed page table. This is in contrast to the *short format* virtual linear page table disscussed previously, which requires less information for each entry. Figure 2.14 illustrates the difference between the two translation entries.

A hashed page table, and particularly the Itanium implementation, has a number of implications.

Firstly, the size of the hash table is variable. Unlike a linear array, where every entry is expected to have a unique home, a hash table is designed to deal with collisions. This means it can have a variable size, probably

roughly based on the amount of physical memory in the system, as this somewhat provides a limit on the amount of address space we are likley to need mapped.

Secondly, the hash function can combine the virtual page number and region ID to make a unique entry, and thus enables the use of a single table for the entire system. This means the entire system can pin a single hash table with a single TLB translation entry; contrast this to the short-format situation where each page of translation entries requires its own TLB translation. A trade-off is that the larger entries for the hashed page table take up more room in the cache; consider we can fit 4 short format entries for every long format entry.

One advantage of the short-format VLPT was that the operating system could keep translations in a multi-level page table, and as long as the leaf entries described a contiguous range of translations, they could be re-used in the VLPT. The short-format translation entry is very practical for this approach, since it mirrors the information an operating system usually keeps in leaf translation entries.

The fact the hash table is pinned with a single TLB entry requires it to be kept as a contiguous source of translation information. The OS must either use the hash table as the primary source of translation entries, or other otherwise keep the hash table in sync with its own translation information.

Fourthly, large-page support is still an issue with the hashed page table. The long format has a explicit *page size* field, so the hardware walker can *load* a translation into the TLB with an arbitrary size (contrast this with the short format, where the information is from taken the default page size for the region).

However, one still has the issue of not knowing the page size when hashing the virtual address. On the Itanium, the hash table index is calculated via the virtual page number[4], preferred page size for the region and the region ID. Thus if a large-page is mapped into a given region, each sub-page (as specified by the region size) must have an entry mapping the larger page. A potential solution is to only put a translation for the first page of a large-page in the hash table and hope that any access to the large-page happens linearly from the start; otherwise the slow path of having the OS deal with the fault must be taken.

If one either pre-fills the sub-pages of a larger page, or fills them lazily on fault, this can create a significant overhead when flushing. Each potential hash table entry must be calculated and purged, which for wildly differing page sizes (say, 64MiB versus a 16KiB page size) becomes a major overhead.

In summary, the long format allows us to reap both the benefits of having more TLB entries available (due to the single pinning) and potential to hardware load large-page entries. The main drawback is the larger cache footprint of the long-format entries.

2.4.5.4 Hardware

As with the UltraSPARC, the Itanium has a TLB hierarchy. The Itanium implements a small L1 TLB which is used for a *prevalidated* L1 cache [MS03, Lyo05]; a unique design which allows a physically-tagged cache with less TLB overhead. A larger general purpose, fully-associative 128 entry L2 TLB is then provided for the slower path.

2.4.6 PowerPC

2.4.6.1 POWER5

The PowerPC architecture is the basis of IBM's high-end POWER5 processor offering. Virtual addressing uses a segmentation scheme where the top parts of a virtual address are looked up in a segment table to give a larger 80 bit virtual address.

As illustrated in Figure 2.15, there are 28-bits reserved for offset within a segment, giving a maximum possible segment size of 2^{28} or 256MiB. The segment descriptor stored in the segment table flags a particular segment as being mapped with base size pages (4KiB) or as being mapped with *large* pages, where large is an implementation-defined size.

For example, the 970FX processor (POWER5) supports segments with either a 4KiB or 16MiB page size. The processor has a unified (instruction and data) 1024 entry 4-way set associative TLB, which is susceptible to

[4] without the top 3 region bits; this way no matter what region an address is mapped into, if they share the region ID they will map to the same hash table index

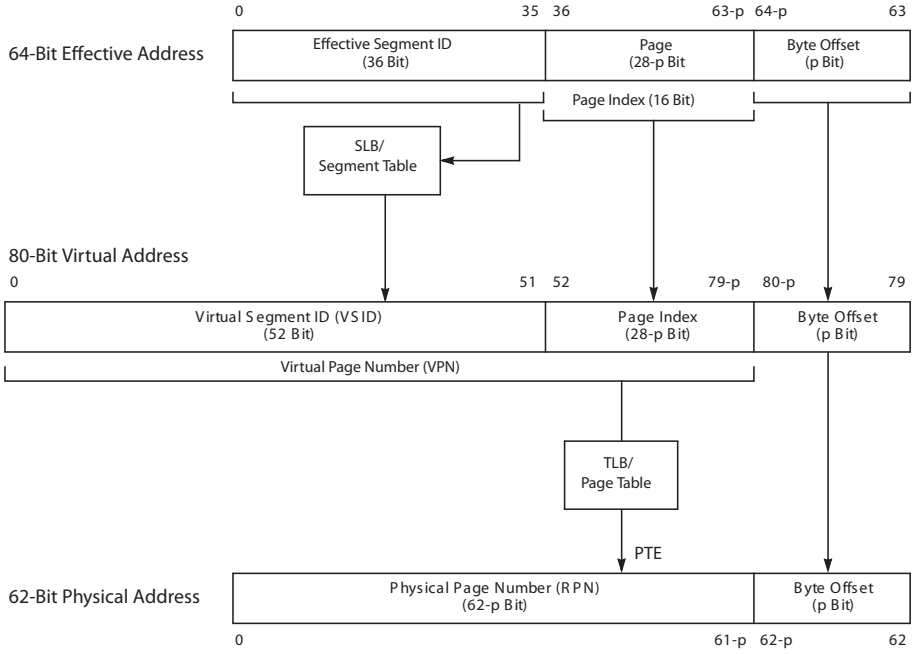


Figure 2.15: Virtual address translation in the PowerPC [IBM05]

aliasing issues. As per Figure 2.15, the size of the page for a virtual address is known *before* TLB lookup, as it is taken from the segment table.

This allows the processor to *hash* the TLB entry such that the two page sizes can co-exist. Future POWER processors will have support for more page sizes.

2.4.6.2 Cell Broadband Processor

The Cell Broadband Engine (CBE) [IBM06] is a new single-chip multiprocessor with discrete processor elements working on shared memory. At the centre is a *Power Processor Element* (PPE) and a number of specialised *Synergistic Processor Elements* (SPE).

Currently the CBE processor has one PPE and eight SPEs, and the PPE is based on a PowerPC design.

The PPE supports 3 large-page sizes; 64KiB, 1MiB and 16MiB. Two of these can be selected via a *large-page* bit within a PTE (the "LP" bit); which two page sizes this bit chooses between is determined by values in a register configured by the OS during setup. The segmentation model as described for the POWER5 is still enforced, and all pages within a segment must be the same size.

As with the POWER5, the processor has a 4-way set associative TLB and uses a hashing scheme to find the way index. Again, the page size is known from the segment descriptor, and effects the hashing function used and the bits matched on the way lookup.

2.4.7 x86

The x86 has been the most prominant architecture for personal computers since the release of the IBM model 5150 in 1981, which was based on an Intel 8088 processor.

The processors memory management has undergone several overhauls in it's lifespan. Originally, the processor was *segmented*, meaning it managed memory in blocks (segments) based on addresses held in *segment registers*. Original implementations used 4 bits to select a segment and 16 bits for offset within that segment, giving a maxium memory of 2^{20} bits or 2MiB. This is illustrated in Figure 2.16.

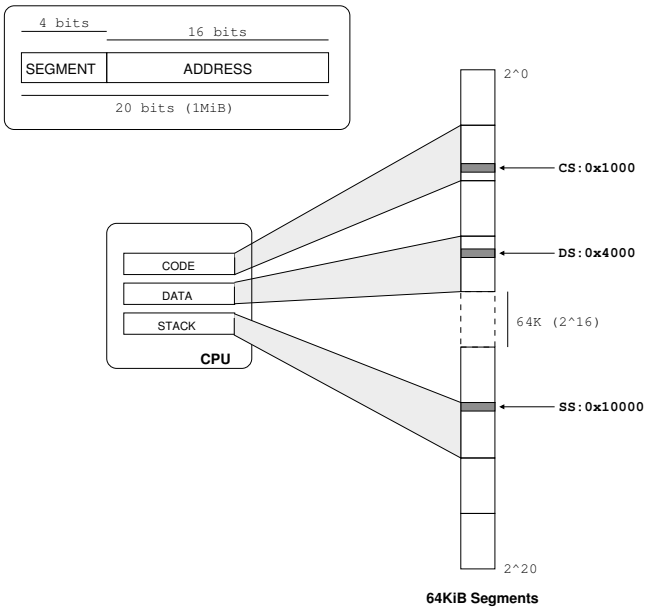


Figure 2.16: Illustration of segmentation

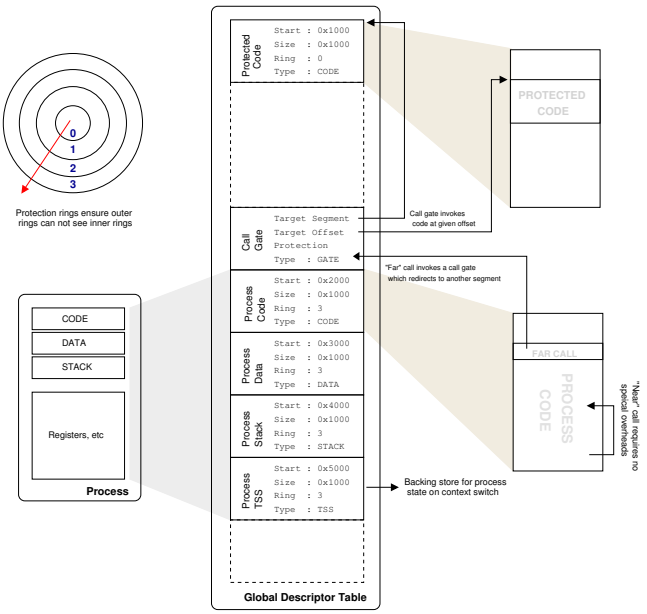


Figure 2.17: Illustration of the post-386 segmentation scheme

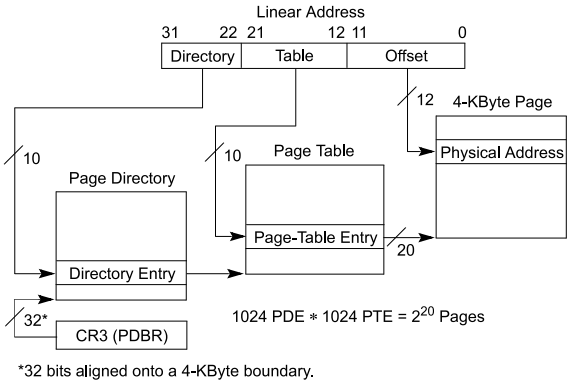


Figure 2.18: IA32 translation [Int01]

A fixed segmentation scheme became far too limiting as the processor grew to support a 32 bit virtual address space with the 386 implementation. The basic concept of segments remained, but in a highly modified version. The underlying concept is that of variable-sized segments, each specified via a *descriptor*.

Segment descriptors are kept in a *global descriptor table*, which all processes have access too (a per-process *local descriptor table* also keeps a range of process private segments). Each of these segments describes a region of virtual address space and an associated protection level for the segment. To enforce protection, a *call gate* is used whenever a thread of execution wishes to call into another segment. This redirects the code to a known location which can allow or deny the request. Although general purpose, the major use is for system calls to the operating system[5]. This model is illustrated in Figure 2.17.

The implementation of this segmented scheme presents one impediment to multiple page size support. Each segment descriptor has a *length* field of 20 bits and a single bit *granularity* flag which defines the segment length as either byte sized (giving a maximum 1MiB segment) or in 4KiB pages (allowing a full 4GiB segment). Different granularity hints may allow for a larger range of page sizes (such as in the Alpha, Section 2.4.1) but the dual problems of their not being enough room in the 64 bit segment descriptor to allocate more bits to granularity, and any modification being a major ABI change.

Since the early Pentium models, the IA32 architecture has implemented *page size extensions* (PSE) to enable a limited range of larger page sizes.

The IA32 architecture implements a hardware based page table walker, hence to avoid duplication overhead the operating system is largely tied to storing virtual address translations in the multi-level format it supports.

We can see from the illustration in Figure 2.18 that the 1024 entries at the page directory entry each point to a page table page that maps 1024 4KiB pages. It is thus no surprise that the additional page size supported by PSE is 4MiB; achieved by turning the page table bits into offset bits.

IA32 only supports these two pages sizes. Large-page entries are kept in a separate TLB [Int01], thus using them has the added benefit of freeing all entries in the smaller TLB (and also avoiding problems with keeping multiple page sizes in the one TLB as previously mentioned).

[5]Modern models implement *fast system calls* which remove the general nature of switching between any segment to a very limited subset. Optimisations appropriate for system calls can then be implemented, leading to a faster return to user code

Processor	Hardware supported page sizes	TLB Load Strategy
Alpha 21164[Sam97]	8KiB, 64KiB, 512KiB, 4MiB	SW(PAL[a])
sun4u (UltraSPARC III)[Sun05b]	8KiB, 64KiB, 512KiB, 4MiB	SW-CW, SW
sun4v (Niagara)[Sun05a, Sun05d, Sun05c]	8KiB, 64KiB, 4MiB, 256MiB	SW-CW, SW
ARM11 (ARM1136JF-S)[ARM05]	4KiB, 64KiB, 1MiB, 16MiB	HW
x86, x86-64[Int01]	4KiB, 4MiB	HW
POWER5 (970FX)	4kb, 16MiB	HW
MIPS R10000	4KiB, 16KiB, 64KiB, 256KiB, 1MiB, 4MiB, 16MiB	SW
Itanium2[Int00]	4KiB, 8KiB, 16KiB, 64KiB 256KiB, 1MiB, 4MiB, 16MiB, 64MiB, 256MiB, 4GiB	HW-CW, SW

- HW refers to a hardware page table walker
- SW refers to a software page table walker
- HW-CW refers to a hardware cache walker traversing a software maintained cache, automatically finding and inserting translations.
- SW-CW refers to a fast path software walker traversing a software maintained cache.
- HW refers to the processor directly walking multi level page table structures.

Table 2.1: Comparison of MMU supported page sizes for common processors.

[a] *Processor Abstraction Layer*. This is a software layer, but is not directly modifiable by the OS

3 Large-page Policy

Operating systems generally deal with memory in only a fixed, single page size. This reduces complexity whenever dealing with pages of memory, since they are always a known size.

To use the multiple page-size support of modern processors, an operating system must provide contiguous virtual mappings to contiguous physical pages.

Contiguous virtual pages are not a primary concern; virtual address spaces are large, sparsely populated and plentiful. Conversely, contiguous physical pages to back these large virtual pages are not plentiful. Compared to virtual address spaces, available physical memory is very small.

Below we categorise and examine some existing approaches to managing these trade-offs.

3.1 Large-page policy approaches

We can divide these polices into three groups [Szm00]

1. **Global policies** used fixed-sized superpages, either preselected before mapping or chosen closer to runtime creation of a mapping. The superpage is never *promoted* (grown) or *demoted* (shrunk).
2. **Static policies** are categorised by a "best effort" promotion and demotion strategy which can grow and shrink page sizes, but will not copy pages to smaller or larger physically contiguous regions to achieve this.
3. **Dynamic policies** implement promotion and demotion; copying frames to be physically contiguous when appropriate.

3.2 Global

3.2.1 Fixed multiple page sizes

Although not directly a superpage technique, the operating system can choose to use as its base page a size greater than the processor's smallest page size. This can have performance improvements due to lower page-fault overheads, but the issues in Section 1.2.2 remain relevant.

3.2.1.1 Page Clustering

Page clustering [Irw03] is a scheme for managing physical memory at larger than base-size granularity.

In earlier Linux versions, each frame of physical memory had a `struct page` descriptor (of around 64 bytes) kept in a direct mapped array. A direct-mapped array becomes a problem as the physical memory to manage grows larger, since the array must grow in a linear fashion.

Page clustering attempts to reduce the size of the array by managing larger than base size frames. For example, if the system manages 8KiB frames rather than 4KiB ones, overall storage requirements are halved.

This raises issues for page table management, since the one–to–one mapping between a page table entry and a `struct page` is removed. Irwin's work modified the Linux kernel to handle the translation between smaller virtual pages and larger page frames.

These problems have a natural corollary in superpages; rather than reducing the number of physical pages we are trying to reduce the number of virtual pages being managed by the system.

Current Linux approaches use a page table to back the frame table (so called "virtual memmap", since the direct mapped array variable name is `memmap`) or allocate the `memmap` amongst nodes of a NUMA system (termed "discontig").

3.2.2 Pinning

Any large mapping unable to change are an excellent opportunity to be pinned with a single, larger TLB entry.

For example, IA64 Linux pins kernel text and data with a single 64MiB page and x86 processors with PSE extensions (see Section 2.4.7) pin kernel data with 4MiB pages.

Pinning is a good approach for statically-sized, known to be frequently re-used code or data. Unfortunately this is relatively rare and so of limited general purpose value. A good general purpose super-page policy would hopefully identify the frequently used area and map it with a large-page, making the pinning superfluous.

3.2.3 HugeTLB

HugeTLB is Linux's current method of utilising large-pages. It was merged in for the `2.6.6-rc1` kernel release around April 2004.

HugeTLB is very much a global approach. The system administrator is responsible for preallocating a range of physical pages which will be assigned to a *HugeTLB region*. The kernel will map these pages with a single administrator-defined page size; obviously accounting for the page sizes supported by the hardware.

Applications can access HugeTLB memory in two ways:

1. Via `mmap` of a file on a special virtual file system of type `hugetlbfs`.
2. Via standard SYSV shared memory calls `shmat` and `shmget`. An extra flag `SHM_HUGETLB` is passed along with the usual information to setup the mapping.

One advantage of this scheme is that the underlying implementation is relatively simple. Since superpages are completely separated from normal pages little change is required to code. Fault paths can simply check if the faulting address lies in a large-page region, and act appropriately[1]. The region can be grown (and shrunk) by the administrator, pending sufficient contiguous physical pages.

The static allocation is often suitable for applications such as databases or scientific applications which allocated large, fixed buffers. However, memory allocated to the HugeTLB region can not be used by applications not modified to use it. This is an extreme form of internal fragmentation and can lead to wastage of memory resources.

Another issue is that only a single large-page size may be used. This is suitable for processors such as IA32 which only support a single larger page, but most other modern hardware provides a range of page sizes.

The scheme is very susceptible to external fragmentation, since there is a race condition between administrators allocating memory for large-pages and other processes in the system. The usual solution is to request the memory very early in the boot process, before many other processes have had a chance to run. Internal fragmentation is also a problem, because physical pages allocated to the HugeTLB region cannot be used for smaller allocations.

The lack of transparency has restricted the use of HugeTLB. Since the `mmap` interface requires an application to know the mount point of the HugeTLB virtual file system, which further requires system administrator intervention to set up, use has been restricted to limited environments. The SYSV shared memory interfaces can make use of the HugeTLB region more easily, but unless an application is sharing memory it is unlikely to use these primitives for memory allocation, so would need to be rewritten.

Currently, developers are working on wrapper libraries to simplify the operation of HugeTLB for programmers[2].

[1] On IA64 this is particularly easy, as a *region* is dedicated to HugeTLB

[2] `http://sourceforge.net/projects/libhugetlbfs`

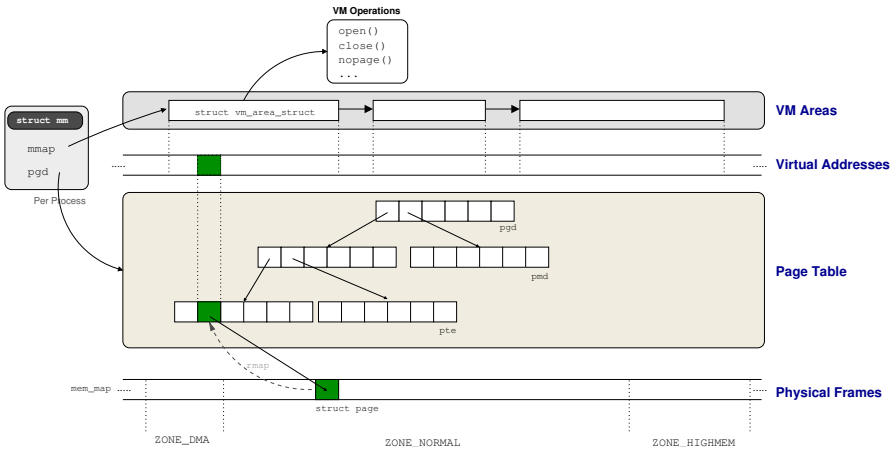


Figure 3.1: Linux memory layout [WSF02]

3.2.3.1 OpenVMS Comparison

OpenVMS has *global section* support which is somewhat similar, if not more advanced than the Linux HugeTLB implementation. Global sections provide support for shared mappings, with shared page tables, to processes. Applications use the "very large machine" (VLM) API to create and utilise global sections.

By themselves, however, global sections did not guarantee the ability to use large-page mappings. OpenVMS previously supported a mode of operation where the kernel could be set to use less than the full available physical memory, and suitably privileged applications could utilise the remaining (physically contiguous) memory, presumably with large-page mappings. This of course has many problems; not least of which is that the operating system no longer has control of the memory so either no sharing of resources occurs, or it happens at another, higher layer.

Thus OpenVMS introduced the concept of *reserved memory regions* [NK98] to back global sections. A reserved memory region is a physically contiguous block, suited to being mapped with large-pages. Since OpenVMS is a system (rather than Linux, which is only a kernel), the reservation process was incorporated into a system registry and is set up at boot time, before other processes have a chance to claim memory. Administrators can control the memory dedicated to reserved memory regions, but unlike the previous scheme they remain under operating system control.

Although Noel and Karkhanis [NK98] do not give detailed performance figures, other papers they cite do show significant improvements from global section (and underlying reserved region) support.

3.2.4 Winwood et al.

Winwood et al. [WSF02] propose a similar scheme for Linux which has not been implemented in the core kernel. They allow a programmer to use the `madvise` system call to modify the page size of a selected region of memory.

The Linux model is illustrated in Figure 3.1. This model was modified to allow each `vm_area` to have its own page size. Firstly, `madavise` was modified to store the page size for an area in the `vm_area` struct.

Each area has a set of VM operations, which are pointers to functions to operate on the area. This allows transparency between files mmaped to disk and memory, for example. The `nopage` operation, called when a page fault is taken, is given as a parameter the preferred page size for the area.

The page table was modified such that each *pte* was flagged with the page size it represented. In a further enhancement, non-leaf entries of the page table are allowed to store a translation entry if they map a region as large as their lower levels (for example, with 4KiB pages and 4 byte PTE entries, a single PMD could map a 4MiB superpage ($\frac{4096}{4}$) This has several advantages

- since each level can represent a page size, there are more ways to represent superpages than the limited number of bits in a PTE entry.
- since the entire tree may not need to be walked, efficiencies can be gained.

The trade-off is large changes to the code base, which has many assumptions about the shape of the page tables, and increased complexity in the page table walking paths.

However, as implemented in the paper, large-page allocations are taken from a separate *largepage* zone. Zones are simply regions of physical memory, each of which can be managed by the kernel in a separate way. The *largepage* zone is sized at boot, similar to HugeTLB memory, and thus categorises this approach as a global one. The *largepage* zone is allocated by the usual Linux buddy allocator.

One motivation for this approach was that in the kernel version used for the paper, there was no effective way to find what *pte*'s might map a physical frame. This meant there was no straight forward way to reduce fragmentation of the large `ZONE_NORMAL` zone. However, current versions of Linux include an `rmap` which has a reverse map from a `struct page` back to a list of the `pte`'s which map it.

They firstly validated their results with a microbenchmark, mapping a heap with both 4K and 4MiB pages and walking it in an adversarial fashion to stress the D-TLB. Their results showed the large-page walk scaling better, since with larger pages less cache entries are taken up by translation entries and are hence available for data.

SPEC CINT2000 workloads were also examined. The `sbrk` system call was modified to map with large pages, and `malloc` instrumented to always use `sbrk` (for large allocations on Linux, large `malloc`s will by default use `mmap`). Overall, performance improved around 15% across all the tests.

Similar to Shimizu (Section 3.3.2) the limited page size support and TLB size of the x86 processor was a constraint.

An interesting line of analysis was Java programs. They suggest that due to JVM's doing just-in-time compilation code and data can end up being in the same memory heap. Code, which displays better locality than data, is probably more suited to smaller pages to avoid wastage, especially important on a platform like x86 where larger TLB entries are a scarce resource.

3.2.5 Solaris

3.2.5.1 Intimate Shared Memory

Large-page support in Solaris 2.6 through Solaris 8 was via a specialised form of System V shared memory referred to as *intimate shared memory* [McD04]. This is similar to the Linux HugeTLB concept (Section 3.2.3); shared memory requested as "intimate" (`shmat()` called with `SHM_SHARE_MMU`) and will be mapped with 4MiB pages where possible (the mappings will be shared by all processes using the mapping, hence the "intimate").

Dynamic ISM (DISM) was added to Solaris 8 (Update 3) to allow dynamic re-sizing of ISM areas; particularly useful for databases which previously required a shutdown-restart cycle to change the size of ISM caches. Solaris 9 expanded ISM to support intermediate large-page sizes.

3.2.5.2 MPSS

MPSS, for *multiple page size support*, was introduced with Solaris 9 as a method for allowing applications to request larger pages without needing to use ISM. Like the HP-UX (Section 3.3.1) and IRIX (Section 3.4.1) schemes, MPSS requires an application (or administrator) to request certain page sizes for the application. MPSS support is available via a number of methods

- The **`mpss.so.1` shared library wrapper** allows setting of page sizes for stack and heap via environment variables.
- **`ppgsz`** is a system utility that allows setting of stack and heap page-size for existing processes.
- The Sun compiler can be passed flags to instrument the binary with page-size information.

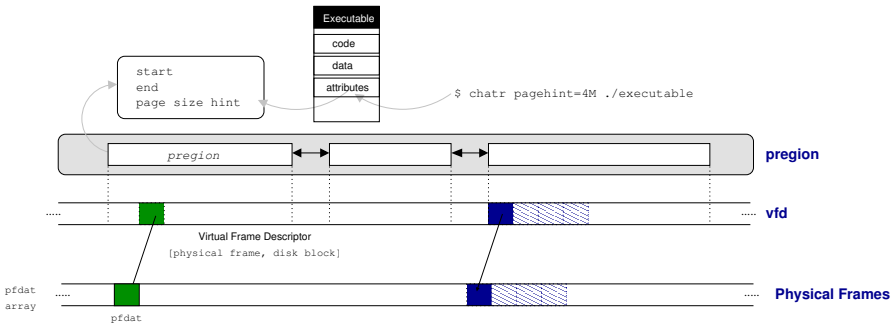


Figure 3.2: A simplified view HP-UX memory management (hardware independent side).

- Applications can be modified to use the **`memcntl`** call to request a larger page size for a specified address range.

MPSS has also been expanded for `Vnodes` (VMPSS); that is text and library code.

The latest work automatically selects page sizes for stack, heap, `mmap`ed memory, text and data based on a simple set of policies, and is known as *MPSS out-of-the-box* (MPSS-OOB) [Low05]. After modelling typical workloads, the TLB abilities of processors are taken into account in creating a policy for automatically requesting larger pages.

MPSS-OOB does not deal with fragmentation [Low05], or explicit promotion or demotion. Future enhancements include anti-fragmentation physical memory allocators, adaptive page sizing algorithms and large-page capable page cache.

3.3 Static

The previous section exaimed a *global* approach where a limited range of fixed page-sizes could be utilised by proceses. Other techniques in both the literature and production allow for a wider selection of page-sizes, often chosen dynamically based on various heuristics. We term these a *static* approach. The static approach is more suitable for our goals of transparency and ability to chose the most correct page size.

In these types of system an initial page-size for a region is chosen based on some heuristics. Static approaches generally support reduction of a region, as this is a requirement for transparency. For example, a common operation is to use the system call `memprotect` to modify the permissions on a region of memory. This splits a single region into two, requiring two TLB entires and consequently a reduced page size.

However, unlike dynamic approaches (discussed in Section 3.4) these approaches do not allow for arbitrary growth of a region into a larger page.

Below we exaimine a number of approaches that can be categorised as a static large-page policy.

3.3.1 HP-UX

Subramanian et al. [SMPR98] implemented multiple page size support for the HP-UX operating system. As illustrated in Figure 3.2, a *page size hint* is suggested by an administrator and added to the attributes of a binary executable. This information is stored in the *pregions* (similar to a Linux *vma*, see Figure 3.1) and can be used to select a page size on fault.

HP-UX will attempt to fulfil this hint unless there is insufficient contiguous memory or the system is coming under memory pressure. The hinting scheme is also supplemented by transparent hinting mechanisms. For example, heap *pregions* which grow to a large size in small increments are tracked, and will have their hints upgraded. Thus an `sbrk` call may receive more memory than is requested (16KiB instead of 4Kib, for example) to utilise a superpage. Hints can also be downgraded under memory pressure to avoid wastage via internal fragmentation.

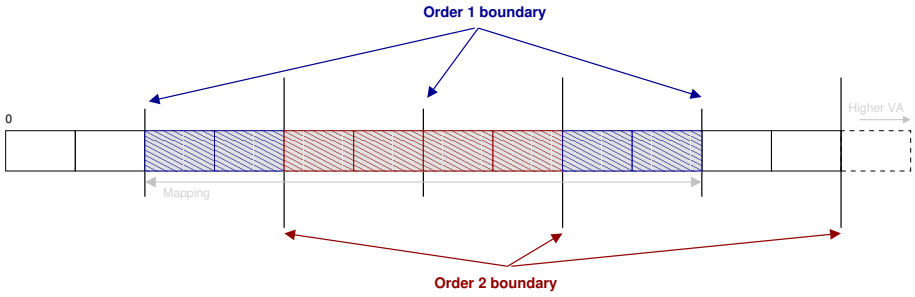


Figure 3.3: Shimizu and Takatori [ST03] size superpages on boundaries, choosing the largest superpage possible within a mapping. Above we see the 8-page mapping is covered by two superpages of order 1, and one superpage of order 2.

Rather than modify all VM structures to handle multiple sized pages, a superpage is defined as a contiguous group of base pages. This reduces modifications required to the VM layers. Page demotion HP-UX is an infrequent operation, but may happen when a mapping is modified (for example, if the protections of a small part of a large-page are modified, it must be split). Another case for demotion is when the pageout daemon wishes to remove a large-page. The page daemon is less aggressive in removing large-pages because they are considered to have higher chance of representing active data; it was suggested more real-world feedback was required on this policy.

Physical memory is allocated with a buddy allocator. Two of free frames are kept, *cached* and *un-cached*, with the cached list being checked last so that it maintains cached data as long as possible. Lists for each possible page size in the buddy allocator are kept, facilitating quick lookup.

The overall results show excellent speedups for a range of benchmarks; in fact some benchmarks run faster than might be expected for less TLB misses – this could be attributed to lower cache pollution.

However, as identified by Szmajda [Szm00], the benchmarks tended to use the maximum superpage size available to them, i.e. many intermediate page sizes were not created, suggesting the benchmarks were mapping large amounts of memory in a machine with no memory pressure. More realistic workloads would be helpful to see the true effects of multiple page size support.

3.3.2 Shimizu and Takatori

Shimizu and Takatori [ST03] propose a *transparent* superpage implementation for Linux on Alpha, Sparc64 and IA32.

Unlike a global approach, superpages are sized at runtime by the operating system; specifically at `mmap` time.

Their approach sizes naturally-aligned superpages as large as possible to cover a given mapping, as illustrated in Figure 3.3.

The frames are marked with their superpage size when the *virtual memory* is allocated by the system. This means until a page is faulted in, there is no memory backing it. At page fault time a suitably large region of contiguous memory needs to be found, this is requested from the standard Linux buddy allocator (see Section 3.3.4).

Shimizu and Takatori do not handle page promotion, but do handle demotion. If sufficient contiguous memory cannot be found when a page is faulted, the superpage will be divided and a smaller allocation requested, until the superpage has been broken down into base page sizes. To attempt to maximise potential superpages, mappings were (optionally) aligned on large superpage boundaries; however this can lead to fragmentation of the virtual address space which may be an issue for 32-bit processors.

Overheads for promotion were considered too high, and it is suggested that by initially covering the mappings with the largest superpage, the mapping starts out with a "best case" scenario. Thus only the demotion case need be covered for when the system is under memory pressure and contiguous memory is not available.

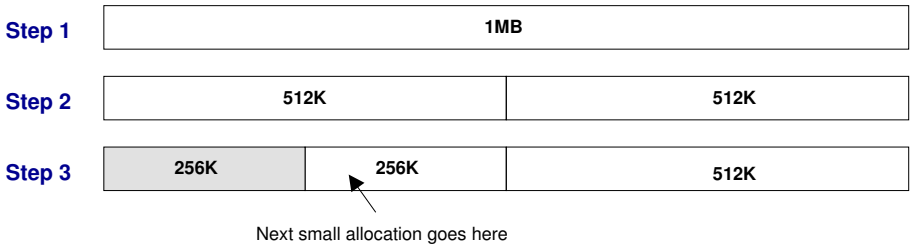


Figure 3.4: A buddy allocator reduces fragmentation, but by packing allocations together reduces the ability to promote allocations to larger superpages.

Performance results are excellent for a limited variation of tests run. A matrix transformation benchmark, which is extremely sensitive to TLB coverage, shows excellent speed-ups using superpages, as expected. However, this test allocates and frees large regions of contiguous memory and will not create significant fragmentation within the system. SPEC results are less impressive, although shows a small speedup.

The results for the x86 processor are limited by insufficient page size options, requiring large alignment and hence larger virtual address space fragmentation. Without making sure allocations happen on a 4MB boundary, no superpages can be mapped. The attempt to always map the largest pages possible may also exacerbate this problem, especially if valuable large entries have wastage due to internal fragmentation.

Some analysis was done of requested contiguity that was unavailable (leading to demotions) but this did not seem like a significant overhead, but this could be due to the limited range of benchmarks run.

3.3.3 Fragmentation Issues

Without specialised hardware support such as a complete sub-blocking TLB, superpages require contiguous physical memory. An operating system generally considers *free* frames as a fully-associative cache; in other words, any free frame is considered valid to back a new mapping [TH94]. Thus to allow effective use of superpages, the operating system memory allocator must take into consideration contiguity of physical frames. A global approach attempts to avoid fragmentation issues by preallocating a large region of memory.

Firstly we discuss some of the issues around fragmentation, then evaluate some approaches.

3.3.4 Buddy Allocators

Buddy allocators [Kno65] allocate memory as a large block, breaking into smaller blocks to satisfy smaller requests. A binary buddy allocator, the simplest implementation, splits into two blocks where are kept as *buddies*. When a block is freed, its buddy (i.e. the block it was split from) is checked to see if it is also free, and if so the areas are merged into a larger contiguous block.

Linux uses a buddy allocator for allocating physical frames [Gor04]. Buddy allocators are also used in systems with large-page support, such as HP-UX [SMPR98].

Buddy schemes have a significant improvement in fragmentation with manageable overheads. However, studies have shown that buddy schemes can lead to large internal fragmentation [WJNB95] and “plateaus” and heavily “peaky” workloads can still lead to significant fragmentation [SMPR98].

As illustrated in Figure 3.4, by compacting memory together, a buddy allocator reduces the ability of allocations to be promoted to superpages.

3.3.5 Slab Allocators

A *slab allocator* [Bon94][3] is a caching allocator for objects.

A slab allocator attempts to speed up allocation for objects which are quickly created and destroyed. It attempts to preserve invariant state of an object between life cycles such that it does not need to be constantly re-created. The allocator will allocate a *slab* of objects which are used as a pool for further object creation.

The slab allocator has several advantages over a simple buddy allocator. In Section 1.2.2.1 we mentioned that fragmentation occurred because objects with dissimilar lifespans were placed near to each other; since the slab allocator allocates like objects together they have similar life cycles. It avoids the situation where a page is "held hostage" by a single object which has not been freed.

Internal fragmentation, an identified problem with the buddy allocator, is also reduced since the size of the slab can be chosen with greater precision.

The slab allocator fails for a general approach, since you must know the size of the object being allocated. This is not the case when the operating system is handing out physical pages to user processes.

3.4 Dynamic

A *dynamic* approach expands on the static and global techniques to handle page-size resizing of aribrary regions. The main challenges to this approach are issues with fragmentation and maintaing efficiency with the increased managment overheads of the dynamic implementation.

3.4.1 IRIX

Ganapathy and Schimmel [GS98][4]propose a general-purpose approach to multiple page sizes, which, like other approaches, attempts to be minimally invasive to the existing operating system VM. Their work is implemented on IRIX.

As in Figure 3.2, IRIX manages physical frames with a `pfdat` structure. As with the HP-UX approach, modifying these structures to map multiple page sizes would require re-architecture of the entire virtual memory subsystem, so `pfdat` structures are extended to have an *order* field indicating what size superpage they are a part of.

Similarly, the upper VM levels are extended to mark individual pages as part of a superpage, again to limit the modifications required.

As mentioned in Section 2.4.2 the MIPS processor has a software-loaded TLB, which includes a *page-size mask* to find the correct entry. Checking multiple page sizes implies a slower TLB miss handler, but the authors were able to implement a per-process TLB handler such that processes not using large pages do not pay the penalty (and, we presume the advantages of large pages outweigh the costs in the other case).

IRIX has an existing *policy* system for virtual address ranges; a policy module can be created and then attached to virtual address regions via system calls. This reflects IRIX's usage on large NUMA machines; for example, policies can control where memory should be allocated in a large NUMA system. The policy is expanded to include page size hints, and policy can dictate that the size is a hint (non-blocking in case of insufficient contiguity) or a requirement (blocking). The system does not do online promotion of superpages, but an application can request upgrading of a memory region via `madvise` system calls. There is also a tool to wrap existing binaries with policies without needing to change source code.

Page Migration moves busy frames to enhance contiguity of memory within the system. This is done in the background by a *coalescing daemon* which has different levels of aggressiveness; *weak* will simply coalesce free pages, *mild* will move pages given a threshold, and *strong* when contiguity is required, such as when a process has made a blocking (i.e. required) large-page request.

Wired frame management attempts to make sure that un-movable kernel pages do not pollute contiguity by keeping them together.

[3]The original author tells a humorous anecdote about deciding on the name *slab* on his blog, available at `http://blogs.sun.com/roller/page/bonwick?catname=%2FSlab+Allocator`.

[4]Presented at the same conference as the HP-UX paper (Section 3.3.1)

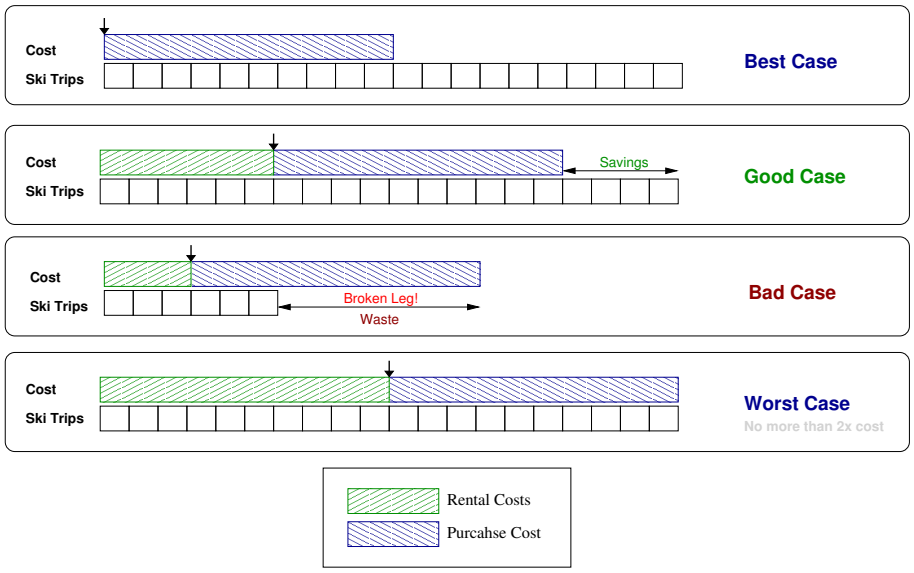


Figure 3.5: The *ski-hire problem* echos issues with page promotion [ROKB95]. When should the skier take the fixed cost of upgrading from renting to purchasing?

Page promotion can be explicitly requested via the `madvise` system call for a region. A large-page region will be allocated and filled via page migration as described above. Online promotion is not done.

3.4.2 Promotion

Dynamic page promotion is not widely implemented in any modern operating system. The schemes presented thus far generally use a larger page at allocation time, and then demote that large-page when required.

Romer et al. [ROKB95] evaluate some techniques for implementing promotion to superpages. The paper compares promotion to the *ski–rental problem*:

> Consider a novice skier. Ski rental is $10 per day, but to purchase the same skis would be $100. Should the skier rent or buy?

An *optimal off line* policy would have the skier purchase the skis if they were sure to ski 10 or more days. However, given the novice can not know this before going skiing, they must use an *online* policy with a *threshold* to decide when to make their purchase. Some complexities of this situation are illustrated in Figure 3.5.

Romer et al. propose a scheme for tracking potential superpage usage and deciding when to promote in an *online* fashion. In summary, the scheme records TLB misses against a superpage that, if mapped, would have prevented them. When a certain threshold of preventable misses is met, the superpage is instantiated in the system ("the skis are bought").

Two counters are kept:

1. A **prefetch** count is increased for a superpage when a miss would have been avoided if that superpage were active.

2. A **capacity** count is calculated from the past stream of TLB misses; if the superpage was active and would have stopped a capacity miss[5] the counter is increased.

Clearly keeping a capacity counter is an expensive proposition; it involves scanning the current TLB entries and coalescing them with a LRU list of pages mapped into the TLB. Romer et al. [ROKB95] give a figure of

[5] A capacity miss happens when the TLB is full, and an entry must be ejected. Thus the TLB would have extra entries free if one large superpage mapping was covering a number of smaller entries.

multiple thousands of cycles per TLB miss; an impractical proposition. Also, the TLB may not be the only factor; for example reloads from the virtually hashed page table would need to be considered.

Thus the authors propose the *APPROX-ONLINE* technique, which only takes into account the much easier to calculate prefetch counters. They show that this scheme performs significantly better than small fixed sized pages, slightly worse than a best-case offline scheme (which has the benefit of hindsight) and almost the same as the significantly more expensive that online with capacity miss calculations.

Fang el al. [FZC+01] revisited the results. The original work by Romer did not take into account reservations (Section 3.4.3 and thus assumed that promotion required a fixed copying overhead. The trace-based measurements of the original paper also does not show external effects such as cache pollution, which are known to increase overheads further. If copying is not required, this is considered a *remapping* case.

Fang et al. produced a analysis using the Impluse system, which implemented a form of no-copy shadow memory superpages (Section 2.2.1.4). As mentioned, a remapping or no-copy scheme is comparable to a reservation scheme which allocates space and allows promotion when suitable.

They found that if remapping is available, then an as-soon-as-possible *ASAP* scheme is desirable. With *ASAP*, a page is promoted as soon as its base pages have been touched. The disadvantage of this scheme is a superpage may be built that is not referenced later (the "broken leg"). They confirmed the results that if copying is required (and thus promotion incurs a large overhead) an *APPROX-ONLINE* scheme is best. Overall, they suggest that more aggressive schemes perform better.

The authors also showed some interesting results for superscalar machines, which were not considered in the original paper. The instructions per cycle (IPC) of a particular application has can affect the relative cost of a TLB miss; if the application has a high IPC then waiting for the low IPC TLB miss handler can waste issue slots that might otherwise be filled, creating more overhead than is reclaimed by the superpages. This re-enforces the concept that larger page sizes are not always a panacea.

Caşcaval et al. [CDSW05] use a system of online and offline *agents* to monitor and analyse program behaviour and determine an optimal page size for an application. This monitoring process is termed *Continuous Program Optimisation*. The system did not provide for a dynamic update of page size, but when restarted the application would get an upgraded page size if decided by the CPO mechanism.

Program memory was categorised into static data (including BSS), small dynamic allocations (below 128KiB) or large dynamic allocations (above 128KiB). By analysing results for executing traces, performance monitoring data and information in the program binary an optimal page size for each type of allocation was chosen. Offline agents do more complex analysis and store results in a database, whilst the online agent makes final decisions about the page size for an application.

One weakness is that the input data may significantly change between a training run monitored by an offline agent and actual input data, possibly leading to bad choices. However, the results generally show a significant reducing in TLB misses and consequent performance improvement.

3.4.3 Reservation

A *reservation* avoids the memory compaction problems as described in Section 3.3.4 by leaving some padding around smaller allocations.

Previously described schemes have managed frames in a binary fashion; either *used* or *free*. Free memory is either chosen on an ad-hoc basis, effectively turning physical memory into a fully associative cache, or bound by a scheme such as the buddy allocator.

Talluri and Hill [TH94] describe a third state for pages — *reserved*. A reserved page is managed by the system, but is known to not contain valid data. Reserved pages have a lower priority for use than *free* pages, so whilst the system is not under memory pressure, the reserved pages will be kept available for promotion. If a mapping grows to cover all the reserved pages it can be promoted to a superpage.

Talluri and Hill describe only a two-page-size system (4KiB and 64KiB) and make any new 4KiB allocation on a 64KiB boundary[6]. As described, only under memory pressure will the reserved pages be used, precluding promotion to a superpage.

[6] Specifically the larger page size is decided by the sub-blocking factor; see Section 2.2.1

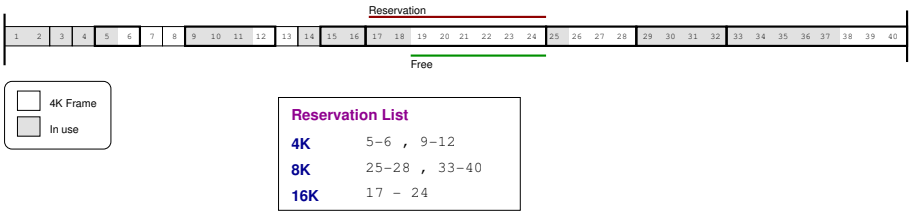


Figure 3.6: An example of the Rice reservation list [Nav04]. For example, should the system decide to make a new 16KiB allocation, the buddy allocator would fail since there is not enough free or unreserved space. We would search the reservation list, which tells us to preempt the 32KiB superpage running from `17–24`. The lists are kept ordered by allocation time, but for simplicity above we show numeric ordering.

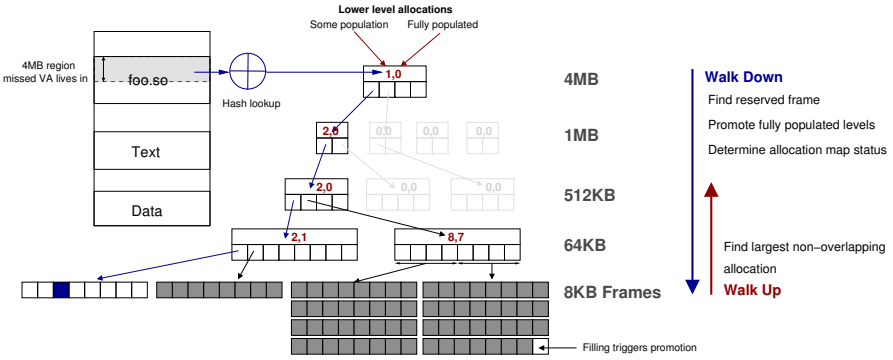


Figure 3.7: An example of a population map [Nav04]. A population map backs a region of address space as large as the largest superpage. As described, it helps with allocation of superpages.

3.4.3.1 Rice Superpages

Navarro el al. [NIDC02] describe a more advanced scheme for allocating memory in a superpage-friendly fashion (we herein refer to this scheme as "Rice superpages" since contributors came from Rice University). Their work targeted the Alpha processor which supports a wide range of page sizes, unlike Talluri and Hill (Navarro's thesis work [Nav04] also targeted IA64 with a similarly wide range of page sizes).

Rice superpages implement a reservation-based system, as described above, but add a unique reclamation scheme which allows easy location of reservations to reclaim under memory pressure.

Firstly the traditional buddy allocator is tried to find a contiguous block for the new reservation. Should this fail, a larger reservation may be *preempted* to reclaim some of its contiguous memory.

3.4.3.1.1 Reservation Lists

To facilitate this, a reservation list is kept for each page size in the system, and reservations are put into the *reservation list* depending on the largest free *extent* they have in their reservation. If the reservation list has no entries for a given size, then the next highest size is tried. Note there is no entry for the largest page size, since there can be no larger pages to split up. The process is illustrated in Figure 3.6.

3.4.3.1.2 Population Maps

The Rice scheme introduces the concept of a *population map* to help to manage common operations with superpages. On each page fault the faulting virtual address is rounded up to the largest superpage size and a hash table referenced to find the population map for the region. The population map is then walked to find the frame in question (illustrated in Figure 3.7).

In the process of walking, we can glean all the information required to manage superpages [Nav04]. Specifically:

1. Map a virtual address to a reserved page frame on fault. By walking down the population map we can see if there is a current reservation for the frame.

2. If there is no reservation for the frame, walking back up the population map can help avoid overlapping frames. The highest upper level with no children is the largest reservation that can be made without overlapping any existing reservations.

3. On new frame allocation the values of *somepop* and *fullpop* are updated; at any level where they become equal a page promotion can be done.

4. When breaking up a reservation, the reservation list needs to be updated. The population table allows reserved regions to be easily classified.

3.4.3.1.3 Issues

Navarro identified some general issues for systems attempting to implement superpages.

Firstly, all modern operating systems use free frames as cache for disk. To maintain reasonable levels of contiguity, these cached pages must be considered available for reservations. However the cached data should remain available as long as possible, hence if a cached page in a reservation is required it should preempt its reservation. This problem tends to become worse over time, as the system fills up the caches.

Another problem is that of *wired* or *pinned* pages, which the kernel will sometimes require. These pages can not be moved, so care must be taken when they are created to keep them together to stop them destroying potential contiguity.

Subsequent work by Navarro with IA64 found some issues with the scheme described above.

Firstly, the depth of the population map grows with the number of available page sizes. As we can see in Table 2.1, Itanium has up to 11 different page sizes; meaning potential for a very long walk process when each page size requires a level within the population map.

Navarro analyses a worst-case sequential allocator; it touches each byte in a mapping sequentially (causing promotions) but never returns to the data. Alpha requires each PTE within a superpage be updated on promotion, thus each page must be traversed for each superpage promotion and then, when freeing, on demotion (with 3 possible page sizes, this means touching each PTE three times on the way up, and 3 times on the way down for a total of six). This means a worst case overhead can be as high as 8.9%, although most common workloads exhibit overheads of 2-3%.

IA64, supporting 7 page sizes in the study, exacerbates the problem and for the same tests shows a worst case slow-down of 32.9%. However the overhead on non-adversary tests was again around 2%. An argument could be made for artificially limiting the potential size of superpages to keep the worst case overheads small. Navarro showed that the penalty imposed by removing potential for intermediate sized superpages outweighed the gains achievable for those applications which exhibit sub-optimal behaviour; only one of the CINT2000 benchmarks had a performance increase of greater than 1% with smaller number of page sizes, but several had decreases, in one case (matrix) running for twice as long.

Secondly, by accessing population maps via a hash of the virtual address, aliasing issues occur. If two processes map the same object on unaligned boundaries they can not share underlying superpages, since the frames can only be correctly aligned for one or the other mapping. When processes might use different areas of the shared object, this introduced both wasted space in reservations and an inability to create superpages as in Figure 3.8. When no base address is given the operating system can choose correct alignments, but explicit starting addresses for `mmap` or mapping from differing file offsets can defeat the scheme.

3.4.3.1.4 Solutions

A *page daemon* normally runs in the background on a system to manage a range of operations on frames of memory. One operation it may undertake is moving inactive pages (those that have not been referenced for a long time) to be available for caching, where they might be more useful to the system. Another common operation is, when under memory pressure, swapping dirty pages to disk freeing them for reuse.

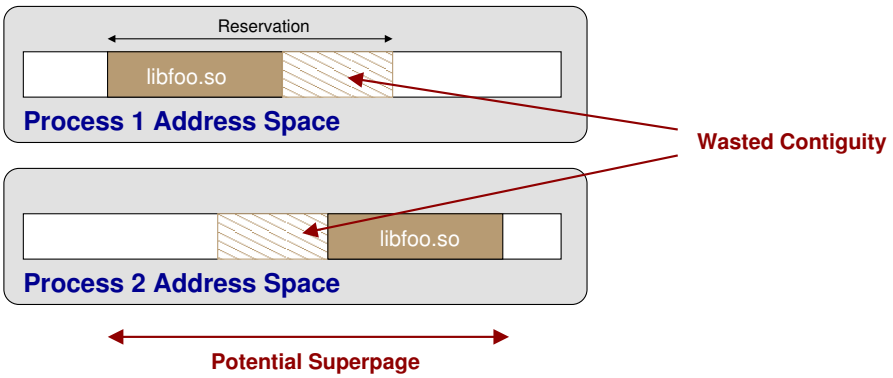


Figure 3.8: Aliasing problems. By not sharing precious contiguity is wasted in unneeded reservations (striped areas) and potential superpage promotions are lost)

Navarro suggests a *contiguity-aware page daemon* which, as the name suggests, extends the operation of the page daemon to attempt to keep as much contiguity as possible. It achieves this by moving inactive pages (those that have not been accessed for a long time) to a cacheable status, which as we mentioned makes them available for reservations. Navarro makes the system more aggressive in marking pages an inactive, meaning faster recirculation time back to cacheable status. Navarro shows that with the contiguity-aware daemon, contiguity over time is greatly increased, which leads to overall increased performance.

Superpage management overheads differ by architecture, but are worse where there are many page sizes to support. A simple static approach is limiting the page sizes available to a reasonably small number, such as 3. This has significant disadvantages; for example if some applications are not provided with the largest 64MiB page size they show slowdowns of up to 47%. In general the penalties far outweigh the benefits.

Navarro found that a *dynamic static* approach worked best, where *each reservation* is given 3 potential page sizes

1. As close as possible to the size of the reservation
2. One size smaller
3. The size between 2 and the smallest page size

Performance for these dynamically chosen three page sizes over a full complement of seven page sizes showed a slowdown of 0–1%.

However, Navarro came up with a number of alternative methods for reducing the overheads whilst keeping a full complement of page sizes available.

For dealing with superpage management overheads, Navarro suggests modifying the reserved page lookup to start from the bottom, rather than the top. Thus each page has a back-pointer to its population map. Clearly this raises a problem when the current page is not part of a reservation, as it will not have a back pointer. However, since FreeBSD keeps pages in a doubly linked ordered list (in fact a *splay* tree) you can easily find the reservations of adjacent pages with simple walks, as illustrated in Figure 3.9.

This removes the requirement for a separate hash table to keep pointers to the top of population maps, as per Figure 3.7.

This covers reserved frame lookup and mapping to regions, but the other role of the population map is in assisting in page promotion and demotion decisions (via the *somepop/fullpop* mechanism). Navarro realised that most allocations happen sequentially, and thus designed a *streamlined population map* which only adds levels as required.

Rather than keeping a record of the "fullness" of a reservations for every possible superpage size (as per Figure 3.7), a tree is dynamically grown to present the present population situation. This is illustrated in

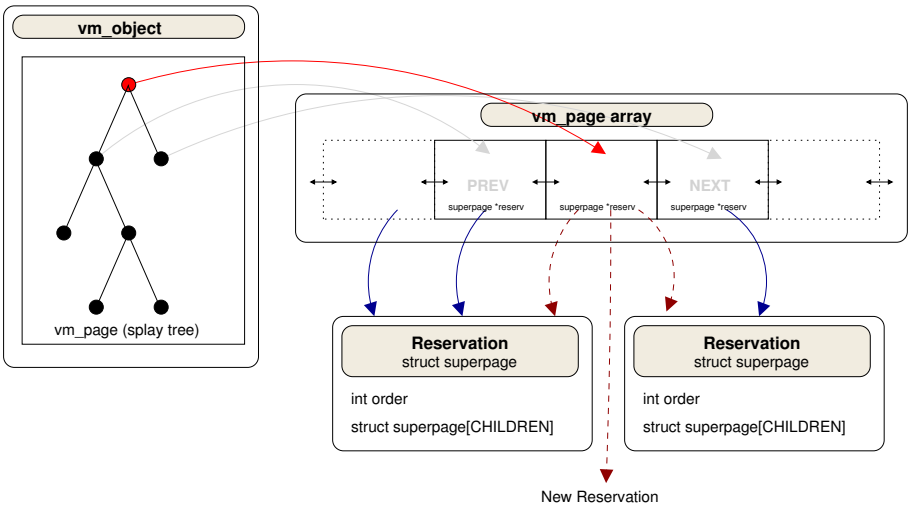


Figure 3.9: By utilising the doubly linked list of pages assigned to an object, a reference can be found to reservations a faulted page might be within. If not, a new reservation can be created that does not overlap. [Nav04]

Figure 3.10. We can see that each node of the tree can keep details of a sequential range of used pages in the reservation; only when a non-sequential allocation occurs are children introduced. This reduces the space and transversal requirements, but still allows easy location of potential superpages.

The final problem is that of updating base frames when their reservation is preempted. If we refer back to Figure 3.9 we can see that if a reservation is preempted and split, each frame allocated to that reservation will need to be updated to point to the new, smaller reservation. To handle this, a *lazy update* scheme is proposed. The old reservation is marked as invalid, but not discarded. The frames are then lazily updated to point to the new, smaller, reservation when they attempt to mark themselves as allocated. A reference counting scheme is provided for the eventual removal of invalid reservations.

These changes significantly reduce the overheads, even over a limited selection of page sizes. Even an adversary case overhead is reduced to a small 2%; other benchmarks below 1%.

3.4.4 Contiguity Daemons

Both IRIX (Section 3.4.1) and Navarro (Section 3.4.3.1) suggest the use of a background daemon to increase contiguity within the system.

Navarro presents a memory compaction algorithm [Nav04]. His results show that a contiguity daemon is a viable approach to restoring contiguity. The results could be improved with the ability to move pages without CPU interaction, as provided on some modern hardware. NUMA considerations were not considered by the work. Modifiable levels of aggressiveness in contiguity reclamation, as done in IRIX, were also not considered.

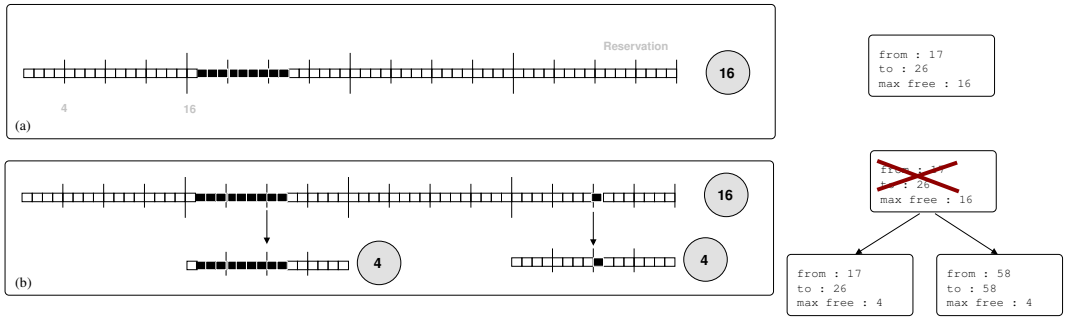


Figure 3.10: A *streamlined* population map of a reservation [Nav04]. The radix tree only grows levels as required. Each node of the tree keeps a start and end pointer to allocated frames within the reservation, and the largest available superpage inside it (illustrated in the grey circle). With sequential allocation, as in (a), there is no need for an extra level to describe the population. In (b) a non-contiguous allocation requires splitting the creation of an additional level. The top level is marked as invalid so we know to descend to children to find the overall population status. This continues recursively.

4 Comparison Summary

Approach	Transparent	Eager	Frame Allocator	Page Table	Sizing Policy	Migration	Explicit Promotion	Online Promotion	Swap
Fixed Larger Pages	✔	✔	N/A	Standard	Static				N/A
Pinning			Reserved	Unmanaged	Static				Unswappable
Linux - HugeTLB		✔	Pre-allocated	Separate	Static				Unswappable
OpenVMS		✔	Pre-allocated		Static				
Linux - Winwood et al.		✔	Pre-allocated	Elastic	Static				
HP-UX		✔	Buddy	Replication	Hinted & Heuristic				Keep
Linux - Shimizu	✔	✔	Buddy	Replication	Largest best fit				Not handled?
IRIX		✔	Buddy	Replication	Hinted	✔	✔		Demote
FreeBSD - Rice	✔		Reservation Lists	Replication	Dynamic		✔	✔	Demote
Solaris - ISM		✔	Buddy		Static				
Solaris - MPSS		✔	Buddy		Static				

- **Eager** refers to allocation of a superpage page being created before there is evidence it will be used.

5 Research Questions and Conclusions

Large pages almost universally provide a performance benefit. However, workload and page size interactions can have large influence over the results. Hence, any general purpose system would need **per-process tunables** such that the page sizes available to the process could be modified at runtime.

Operating systems have not been designed to support multiple page sizes, and thus large-page support must be "added on" rather than "designed in". PTE replication as a basis for superpage support allows minimal modifications to the page table layers, and has been the basis of successful implementations.

Transparency is required to support superpages without address space restrictions or API/ABI changes.

Physical memory allocation is largely dealing with fragmentation (Section 1.2.2.1). Fragmentation has been around for as long as virtual memory has, but superpages exacerbate the problems. There are a number of important interactions to consider:

- **Pre-allocation** and **reservation** schemes allow increased exploitation of contiguity.
- Unused memory needs to be used as **page cache**; reservations should not preclude this.
- **Wired pages** need to be managed in some consistent manner to keep them from polluting contiguity.

Physical memory is usually allocated via a buddy allocator (Section 3.3.4). Often multiple free lists are kept for each page size. Rice reservation lists expand page states from "free" and "used" to include "reserved"; reserved areas can be broken down for allocations if required.

With transparent large pages **demotion** is a critical requirement, since applications may change protection information on smaller boundaries than a region is currently mapped with. Policy around when to demote pages when not strictly required (e.g. swap, out of memory conditions) is less clear; there is certainly an argument for making it tunable.

Promotion of base pages to superpages is a less clear proposition (Section 3.4.2. Fang et al. built on Romer et al. to show that the overhead of detailed statistics was unlikely to match a simpler scheme of promotion once pages were touched. Shimizu (Section 3.3.2) and HP-UX (3.3.1) makes available the largest pages possible to cover a mapping, and then supports demotion should it be required.

IRIX implements a **coalescing daemon** to increase contiguity (Section 3.4.1). This interacts with the memory policy mechanisms and page migration mechanisms to find contiguity as aggressively as required. Navarro presents and analyses an algorithm for coalescing, finding it a viable option for returning contiguity to the system.

Online Promotion is thoroughly covered by the Rice work (Section 3.4.3.1). It needs to be backed by a reservation scheme to avoid fragmentation problems or excessive copying on promotion. **Reservations** pre-allocate an area of memory for a superpage, but can have considerable management overheads, especially as the number of possible page sizes rises. Statically limiting the number of available page sizes is a sub-optimal approach, and more innovative management structures can remove much of the overheads.

The most popular x86 processor is not a particularly good target for superpages. The lack of page sizes means large alignment constraints and virtual address space fragmentation (particularly an issue with a smaller 32 bit address space). The size of the large-page TLB is a bottleneck.

5.1 Research Directions

Transparent large pages are a worthy goal for Linux.

The Linux development model is based around iterative change, and hence any approach should be developed in a piecemeal fashion of separate, but interacting components.

HugeTLB is not going to be removed. Any solution should consider the possibility that the HugeTLB API remains, and is backed by components of the new solutions.

Concurrency, and NUMA concerns in particular, are first class citizens for Linux development. Andrew Morton has suggested big wins in one class can make up for small penalties in another, should it be justified. Any potential work needs to take these concerns into account.

A first implementation must use the existing Linux page tables. Two options are the Winwood approach of keeping page sizes on a per-VMA region, or as has been done on other OS's, PTE replication.

The Winwood approach adds potentially large overhead to the fault handling fast-path of the VM system. A fundamental question is if a VMA really needs to know what page sizes are used within it; this probably has a role more as a hinting mechanism than for management use.

Thus the research should focus on PTE replication. Specifically, the goals are

- Modifying a PTE to have a page size; considering issues such as is there enough room to do this on all architectures, and how much does adding this cost. On Itanium, we must measure the overheads of alternate page table walkers to quantify this cost.
- Modifying relevant fault handlers to recognise this size, e.g. when zapping.
- Coming up with some form of common API so the VM layer can do this for each architecture.
- Handle demotion. `mprotect`, `munmap`, copy on write are all going to break superpages.
- Modifying the guts of each architecture to actually get larger page sizes into the TLB

Once there is the ability to make large pages, we must decide when to make them.

- Anonymous `mmap` is a good place to start. This should catch large `mallocs`; an instant win
- Supporting `brk` and the stack. Something like HP-UX watching for growing areas and mapping larger pages or something
- Enable hinting or policy control? A `/proc` tunable bitmap of acceptable order page sizes may be appropriate, or embedding in a special header in the binary. A wrapper program/environment variables should also be considered.
- Should policy be able to demand a page size. How do we implement this if we can not find sufficient contiguity?

Physical memory is clearly important, but can be separated from the upper layers. All we really need is `alloc_pages(order)`. To avoid fragmentation issues and destroying page cache the physical allocator is going to have to do something smart; either with reservations or some other scheme.

A background coalescing daemon seems to work well. Work on page migration is already happening and seems like a logical fit. This process could be largely orthogonal to VM large-page support, although we may need ways to request the daemon provide us with the contiguity.

Bibliography

[ARM05] ARM Ltd. *ARM1136JF-S and ARM1136J-S Technical Reference Manual*, R1P1 edition, 2005.

[BMS02] David Bradley, Patrick Mahoney, and Blane Stackhouse. The 16KB single-cycle-read-access cache on a next-generation 64b Itanium microprocessor. In *International Solid-State Circuits Conference*, pages 110–111. IEEE, February 2002.

[Bon94] Jeff Bonwick. The slab allocator: An object-caching kernel memory allocator. In *USENIX Technical Conference*, Boston, MA, USA, Winter 1994.

[CDSW05] Călin Caşcaval, Evelyn Duesterwald, Peter F. Sweeney, and Robert W. Wisniewski. Multiple page size modeling and optimization. In *Proceedings of the 14th International Conference on Parallel Architectures and Compilation Techniques*, pages 339–349, September 2005.

[Com99] Compaq. *Alpha 21264 Microprocessor Data Sheet*, 1999.

[CWH03] Matthew Chapman, Ian Wienand, and Gernot Heiser. Itanium page tables and TLB. Technical Report UNSW-CSE-TR-0307, School of Computer Science and Engineering, University of NSW, Sydney 2052, Australia, May 2003.

[Den68] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11:323–333, 1968.

[Den70] Peter J. Denning. Virtual memory. *ACM Computing Surveys*, 2:154–189, 1970.

[FZC+01] Zhen Fang, Lixin Zhang, John B. Carter, Wilson C. Hsieh, and Sally A. McKee. Reevaluating online superpage promotion with hardware support. In *Proceedings of the 7th IEEE Symposium on High-Performance Computer Architecture*, page 63, 2001.

[GCC+05] Charles Gray, Matthew Chapman, Peter Chubb, David Mosberger-Tang, and Gernot Heiser. Itanium — a system implementor's tale. In *Proceedings of the 2005 USENIX Technical Conference*, pages 264–278, Anaheim, CA, USA, April 2005.

[Gor04] Mel Gorman. *Understanding the Linux Virtual Memory Manager*. Prentice Hall PTR, Upper Saddle River, NJ, USA, 2004.

[GS98] Narayanan Ganapathy and Curt Schimmel. General purpose operating system support for multiple page sizes. In *Proceedings of the 1998 USENIX Technical Conference*, New Orleans, USA, June 1998.

[HS84] Mark D. Hill and Alan Jay Smith. Experimental evaluation of on-chip microprocessor cache memories. In *Proceedings of the 11th International Symposium on Computer Architecture*, pages 158–166, New York, NY, USA, 1984. ACM Press.

[IBM05] IBM. *PowerPC Microprocessor Family: The Programming Environments Manual for 64-bit Microprocessors*, 3.0 edition, July 2005.

[IBM06] IBM. *Cell Broadband Engine Progamming Handbook*, 1.0 edition, April 2006.

[Int99] Intel Corp. *Intel StrongARM SA-1100 Microprocessor Developer's Manual*, August 1999.

[Int00] Intel Corp. *Itanium Architecture Software Developer's Manual Volume 2: System Architecture*, January 2000. `http://developer.intel.com/design/itanium/family`.

[Int01] Intel Corp. *IA-32 Architecture Software Developer's Manual Volume 3: System Programming Guide*, 2001. URL `ftp://download.intel.com/design/Pentium4/manuals/245472.htm`.

[Irw03] William L. Irwin. A 2.5 page clustering implementation. In *Proceedings of the Linux Symposium*, Ottawa, Canada, 2003.

[JM97] Bruce Jacob and Trevor Mudge. Software-managed address translation. In *Proceedings of the 3rd IEEE Symposium on High-Performance Computer Architecture*, pages 156–167, 1997.

[Kno65] Kenneth C. Knowlton. A fast storage allocator. *Communications of the ACM*, 8(10):623–624, 1965.

[KP06] Dave Kleikamp and Badari Pulavarty. Efficient use of the page cache with 64 KB pages. In *Proceedings of the Linux Symposium*, volume 2, pages 65–70, 2006.

[Lie96] Jochen Liedtke. *On the Realization Of Huge Sparsely-Occupied and Fine-Grained Address Spaces*. Oldenbourg, Munich, Germany, 1996.

[Low05] Eric Lowe. Automatic large page selection policy. OpenSolaris project Muskoka, Sun Microsystems, March 2005. `http://www.opensolaris.org/os/project/muskoka/virtual_memory`.

[Lyo05] Terry L. Lyon. Method and apparatus for updating and invalidating store data. US Patent 6920531, 2005. Assignee: Hewlett-Packard Development Company, L.P., Houston, TX(US); filed Nov 4, 2003.

[McD04] Richard McDougall. Supporting mulitple page sizes in the Solaris operating system. Sun Blueprints Online, Sun Microsystems, March 2004.

[MCY97] Randy Martin, Yung-Chin Chen, and Ken Yeager. *MIPS R10000 Microprocessor User's Manual, Version 2.0*. MIPS Technologies, Inc., Mountain View, California, 1997.

[ME02] David Mosberger and Stéphane Eranian. *IA-64 Linux Kernel: Design and Implementation*. Prentice Hall, 2002.

[MS02] A.H. Mohamed and A. Sagahyroon. A scheme for implementing address translation storage buffers. In *Canadian Conference on Electrical and Computer Engineering*, volume 2, pages 626–633, 2002.

[MS03] Cameron McNairy and Don Soltis. Itanium 2 processor microarchitecture. *IEEE Micro*, 23(2):44–55, 2003.

[Nav04] Juan E. Navarro. *Transparent operating system support for superpages*. PhD thesis, Rice University, Houston, Texas, April 2004.

[NIDC02] Juan Navarro, Sitaram Iyer, Peter Druschel, and Alan Cox. Practical, transparent operating system support for superpages. In *Proceedings of the 5th USENIX Symposium on Operating Systems Design and Implementation*, Boston, MA, USA, December 2002.

[NK98] Karen L. Noel and Nitin Y. Karkhanis. OpenVMS Alpha 64-bit very large memory design. *Digital Technical Journal*, 9(4):33–48, 1998.

[Pot99] Daniel Potts. L4 on uni- and multiprocessor Alpha. BE thesis, School of Computer Science and Engineering, University of NSW, Sydney 2052, Australia, November 1999. Available from publications page at `http://www.disy.cse.unsw.edu.au/`.

[ROKB95] Theodore H. Romer, Wayne H. Ohllrich, Anna R. Karlin, and Brian N. Bershad. Reducing TLB and memory overhead using online superpage promotion. In *Proceedings of the 22nd International Symposium on Computer Architecture*, pages 176–87, Santa Margherita Ligure, Itay, June 1995. ACM.

[Sam97] Samsung Electronics. *21164 Alpha Microprocessor Hardware Reference Manual*, 1997.

[Sez93] André Seznec. A case for two-way skewed-associative caches. In *Proceedings of the 20th International Symposium on Computer Architecture*, pages 169–178, 1993.

[Sez04] André Seznec. Concurrent support of multiple page sizes on a skewed associative TLB. *IEEE Transactions on Computers*, 53(7):924–927, 2004.

[SMPR98] Indira Subramanian, Cliff Mather, Kurt Peterson, and Balakrishna Raghunath. Implementation of multiple pagesize support in HP-UX. In *Proceedings of the 1998 USENIX Technical Conference*, New Orleans, USA, June 1998.

[SSC98] Mark Swanson, Leigh Stoller, and John Carter. Increasing TLB reach using superpages backed by shadow memory. In *Proceedings of the 25th International Symposium on Computer Architecture*, pages 204–213. ACM, 1998.

[ST03] Naohiko Shimizu and Ken Takatori. A transparent Linux super page kernel for Alpha, Sparc64 and IA32: reducing TLB misses of applications. *SIGARCH Computer Architecture News*, 31(1):75–84, 2003.

[Sun05a] Sun Microsystems Inc., Santa Clara, CA, USA. *The UltraSPARC Architecture 2005*, 2005. `http://www.sun.com/processors/documentation.html`.

[Sun05b] Sun Microsystems Inc., Santa Clara, CA, USA. *The UltraSPARC III Processor User's Manual*, 2005. `http://www.sun.com/processors/documentation.html`.

[Sun05c] Sun Microsystems Inc., Santa Clara, CA, USA. *The UltraSPARC T1 Hypervisor API Specification*, 2005. `http://opensparc.sunsource.net/nonav/opensparct1.html`.

[Sun05d] Sun Microsystems Inc., Santa Clara, CA, USA. *The UltraSPARC T1 supplement to UltraSPARC Architecture 2005*, 2005. `http://opensparc.sunsource.net/nonav/opensparct1.html`.

[Szm00] Christan Szmajda. Virtual memory performance. (Draft Copy), July 2000.

[Tal95] Madhusudhan Talluri. *Use of Superpages and Subblocking in the Address Translation Hierarchy*. PhD thesis, University of Wisconsin-Madison Computer Sciences, 1995. Technical Report #1277.

[TH94] Madhusudhan Talluri and Mark D. Hill. Surpassing the TLB performance of superpages with less operating system support. In *Proceedings of the 6th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 171–182, San Jose, CA, USA 1994.

[TKHP92] Madhusudhan Talluri, Shing Kong, Mark D. Hill, and David A. Patterson. Tradeoffs in supporting two page sizes. In *Proceedings of the 19th International Symposium on Computer Architecture*. ACM, 1992.

[Tzo89] Shin-Yuan Tzou. Software mechanisms for multiprocessor TLB consistency. Technical Report UCB/CSD-89-551, EECS Department, University of California, Berkeley, 1989.

[WEG+86] David A. Wood, Susan J. Eggers, Garth Gibson, Mark D. Hill, Joan M. Pendleton, Scott A. Ritchie, George S. Taylor, Randy H. Katz, and David A. Patterson. An in-cache address translation mechanism. In *Proceedings of the 13th International Symposium on Computer Architecture*, pages 358–365, 1986.

[WH00] Adam Wiggins and Gernot Heiser. Fast address-space switching on the StrongARM SA-1100 processor. In *Proceedings of the 5th Australasian Computer Architecture Conference*, pages 97–104, Canberra, Australia, January 2000. IEEE CS Press.

[Wig03] Adam Wiggins. A survey on the interaction between caching, translation and protection. Technical Report UNSW-CSE-TR-0321, School of Computer Science and Engineering, University of NSW, Sydney 2052, Australia, August 2003.

[WJNB95] Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles. Dynamic storage allocation: A survey and critical review. In *Proceedings of the International Workshop on Memory Management*, pages 1–116, London, UK, 1995. Springer-Verlag.

[WSF02] Simon Winwood, Yefim Shuf, and Hubertus Franke. Multiple page size support in the Linux kernel. In *Ottawa Linux Symposium*, Ottawa, Canada, June 2002.

[WWTH03] Adam Wiggins, Simon Winwood, Harvey Tuch, and Gernot Heiser. Legba: Fast hardware support for fine-grained protection. In *Proceedings of the 8th Asia-Pacific Computer Systems Architecture Conference*, Aizu-Wakamatsu City, Japan, September 2003. Springer Verlag.

[ZFP+01] Lixin Zhang, Zhen Fang, Mide Parker, Binu K. Mathew, Lambert Schaelicke, John B. Carter, Wilson C. Hsieh, and Sally A. McKee. The impulse memory controller. *IEEE Transactions on Computers*, 50(11):1117–1132, 2001.

Copyright (C) 2006 The University of New South Wales